Tutorial I

Comparación entre JavaScript, TypeScript junto a sus librerías, frameworks y herramientas

Taller de Innovación y Emprendimiento Tecnológico

Universidad Adolfo Ibáñez

Profesor: Martín Céspedes Escuti

Ayudante Instructor: Artemio Araya Day

17 de octubre de 2026

Índice

Objetivo 5

Definiciones 6

Android 6

Android Studio 6

Software Development Kit 6

Java Development Kit (JDK) 8

Gradle 10

Sync, Build, Run en Android Studio 12

2 Métodos para Desarrollar una Aplicación móvil 16

Método I: Utilizar HTML, CSS, JavaScript con AppView para visualización y Kotlin para acceder a las funciones nativas del dispositivo. 18

Método II: Utilizar React Native con TypeScript para desarrollar interfaces con componentes nativos así como acceder a las funciones del dispositivo móvil Android 19

Tabla comparativa entre los Métodos I y II 20

Sección de Preguntas 23

¿Por qué se utiliza la librería React.js de JavaScript? . . . 23

¿Por qué utilizar Kotlin y no Java? 24

¿Cuando conviene usar JavaScript sin React.js para desarrollar la aplicación? 28

¿Qué diferencia hay entre React.js, React Native y React DOM? 29

¿Qué lenguajes de programación se utilizan para React Native y conjuntamente con él? 31

¿Que diferencia hay entre WebView y React Native ? 34

Descripción General de la Aplicación Móvil 36

Compilación e instalación de la aplicación móvil 37

Funcionamiento de la aplicación móvil 44

Estructura del Repositorio de GitHub 48

Uso de cada lenguaje en la aplicación 51

Bibliotecas y API utilizadas desde Kotlin 53

Bibliotecas de JavaScript 55

Tutorial 56

Material para desarrollar la Aplicación .apk 56

Descargar e instalar Android Studio 57

Descargar el repositorio de GitHub 58

Abrir el proyecto en Android Studio 59

Instalación y configuración del JDK y sus componentes en Android . 60

Sincronizar el proyecto 60

Generar el APK de la calculadora 62

Encontrar y guardar el archivo correcto 63

Referencias 65

Proyecto de referencia:

artemioadaysolvers/Tutorial-App-I (?).

Leguajes principales:

- JavaScript, HTML y CSS para sistema de visualización con WebView

- Kotlin para acceder a las funciones nativas del dispositivo Android

Principales Herramientas sobre los Lenguajes

- three.js

Objetivo

Este tutorial explica cómo preparar y compilar una calculadora para Android utilizando Kotlin, JavaScript, HTML y CSS.

El proceso comienza con la instalación de Android Studio en Windows y la descarga de los archivos del proyecto desde GitHub, continúa con la configuración de las herramientas de compilación y termina con la instalación de la aplicación en el dispositivo móvil de Android.

La calculadora utiliza HTML para definir la estructura de la interfaz gráfica, organizando elementos como la pantalla y los botones; CSS para establecer su apariencia visual, incluyendo colores, dimensiones, tipografías y distribución de los elementos; JavaScript para implementar la lógica de funcionamiento, procesar las pulsaciones de los botones, realizar las operaciones matemáticas y actualizar los resultados; y Kotlin acceder a las funcionalidad nativas del dispositivo.
, configurar el componente WebView que permite visualizar y ejecutar la interfaz web dentro de la aplicación y facilitar su integración con las funcionalidades del sistema operativo.

Definiciones

Android

Android es un sistema operativo de código abierto basado en el núcleo de Linux, diseñado para diversos tipos de dispositivos. Su arquitectura integra un entorno de ejecución, bibliotecas, servicios y un marco de desarrollo que permiten administrar los recursos del dispositivo y ejecutar aplicaciones (Google, 2025a).

Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para crear aplicaciones para Android. Está basado en IntelliJ IDEA e integra herramientas para escribir código, compilar aplicaciones, ejecutarlas en un emulador o dispositivo físico, realizar pruebas y detectar errores (Google, 2026d).

Android Studio está desarrollado utilizando Java y Kotlin sobre la plataforma IntelliJ de JetBrains (Google, s. f.).

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) creado por JetBrains que reúne herramientas para escribir, ejecutar y depurar código, principalmente en Java y Kotlin (JetBrains, s. f.).

Software Development Kit

¿Qué es el Software Development Kit?

Es un conjunto de herramientas y componentes necesarios para desarrollar aplicaciones para Android. Incluye herramientas de compilación, herramientas de comunicación y depuración con dispositivos, y las plataformas de Android necesarias para compilar las aplicaciones (Google, 2026a).

Se instala en tu computadora y Android Studio lo utiliza para desarrollar, compilar, probar y depurar la aplicación (Google, 2026a).

La compilación de una aplicación en Android Studio se realiza mediante Gradle y el complemento de Android para Gradle, que coordinan el uso de los compiladores y las herramientas del Software Development Kit de Android. Entre estas herramientas se encuentra D8, que transforma el código de bytes Java en código de bytes DEX, ejecutable en dispositivos Android (Google, 2026c) (Google, 2025c).

Durante el proceso de compilación de una aplicación Android, el código fuente escrito en Kotlin o Java se transforma inicialmente en código de bytes para la máquina virtual de Java (Google, 2026c).

Posteriormente, herramientas como D8 convierten este código al formato DEX (*Dalvik Executable*), que puede ejecutarse en dispositivos Android (Google, 2025c). Finalmente, el sistema de compilación empaqueta los archivos DEX junto con los recursos y el manifiesto de la aplicación en un archivo APK, que, una vez firmado, permite instalar la aplicación en un dispositivo (Google, 2025b) (Google, 2026b).

El Software Development Kit sirve para desarrollar y compilar la app para obtener el .apk (Google, 2026a) (Google, 2026c).

- **Para desarrollarla:** proporciona las API que utilizamos desde Kotlin para crear la pantalla, mostrar el WebView y acceder a funciones de Android (Android Developers, s. f.-a, s. f.-e).

- **Para compilarla:** incluye herramientas que Gradle utiliza para preparar los recursos y construir el APK (Android Developers, 2025).

Java Development Kit (JDK)

El **JDK** (*Java Development Kit*, o kit de desarrollo de Java) es un entorno de desarrollo que reúne herramientas para crear, compilar, ejecutar y probar programas escritos en el lenguaje Java (Oracle, s. f.).

Incluye el compilador javac, que transforma archivos de código Java en archivos de clases; bibliotecas que proporcionan funciones reutilizables; y la máquina virtual Java, o JVM, que ejecuta programas preparados para ese entorno. Cada componente cumple una función concreta: el compilador traduce el código, las bibliotecas aportan funciones y la JVM permite ejecutar los programas. En el desarrollo de esta aplicación Android, el JDK proporciona el entorno donde se ejecutan Gradle y otras herramientas de construcción en el computador. Por eso es necesario disponer de un JDK compatible aunque los archivos de la aplicación estén escritos en Kotlin. El JDK seleccionado para Gradle puede ser diferente del entorno Java incluido con Android Studio para ejecutar el propio editor (Android Developers, 2026g; , s. f.).

La relación entre **Kotlin y el JDK** se establece mediante las herramientas utilizadas para compilar el proyecto. Kotlin es el lenguaje en el que se escriben clases como CalculatorEngine, AppView y MainActivity; su código se traduce mediante el compilador de Kotlin, incorporado al proceso de construcción por el complemento de Kotlin para Gradle. En esta aplicación, el compilador produce código de clases para la JVM, que posteriormente se transforma al formato utilizado por Android. El compilador javac del JDK compila código Java, mientras que los archivos .kt corresponden al compilador de Kotlin. Sin embargo, el JDK proporciona el entorno Java donde se ejecutan Gradle y el compilador de Kotlin durante la construcción. Además, Kotlin puede utilizar determinadas bibliotecas y tipos de Java. La selección del JDK y la opción jvmTarget tienen funciones distintas: la primera establece el entorno que ejecuta las herramientas, mientras que la segunda indica la versión del código JVM que debe producir el compilador (Android Developers, 2026g; JetBrains, 2026).

El **SDK de Android** (*Software Development Kit*, o kit de desarrollo de software de Android) reúne componentes específicos para construir aplicaciones des-

tinadas a ese sistema operativo. Su diferencia con el JDK está en su propósito: el JDK proporciona el entorno y las herramientas de la plataforma Java, mientras que el SDK de Android aporta las APIs y utilidades específicas de Android. Dentro del SDK, **SDK Platform** proporciona las APIs utilizadas como referencia al compilar, por ejemplo, las definiciones de clases como Activity, WebView e Intent; **SDK Build-Tools** contiene utilidades empleadas durante la construcción, como las relacionadas con la alineación y firma del APK; y **SDK Platform-Tools** incluye ADB, que permite comunicarse con el teléfono para instalar y probar la aplicación. Estos paquetes cumplen funciones diferentes y tienen sus propias versiones. Por ello, SDK Platform 36 y Build-Tools 35.0.0 pueden participar en una misma construcción: sus números identifican componentes diferentes (Android Developers, 2026g, 2026h, 2026j).

En **esta aplicación**, los componentes se relacionan mediante un proceso coordinado por Gradle: el JDK permite ejecutar las herramientas de construcción; el complemento de Kotlin compila las clases que implementan los cálculos y las funciones nativas; y Android Gradle Plugin incorpora las tareas que procesan los recursos, preparan el código para Android y generan el paquete instalable. SDK Platform 36 proporciona las APIs contra las que se comprueban las referencias del código, mientras que los recursos HTML, CSS, JavaScript y GLB se incorporan al APK para utilizarlos durante la ejecución. Después, ADB permite instalar el paquete en el teléfono y consultar información para investigar errores. El JDK permanece en el computador de desarrollo: el teléfono ejecuta el código DEX mediante el entorno de Android denominado ART y utiliza la WebView para presentar la interfaz web. Por tanto, el JDK y el SDK de Android se complementan durante la construcción, pero aportan componentes distintos. El teléfono necesita una versión compatible de Android para ejecutar la app; su usuario no necesita instalar el JDK ni las herramientas del SDK (Android Developers, 2026f; Android Open Source Project, 2026).

Gradle

Gradle es una herramienta de código abierto destinada a la automatización de los procesos de construcción de software (*build automation*), que permite definir, configurar y ejecutar las operaciones necesarias para transformar un proyecto de código fuente en uno o varios productos de software. Según su documentación oficial, Gradle organiza estos procesos mediante proyectos (*projects*), tareas (*tasks*), dependencias (*dependencies*), complementos (*plugins*) y archivos de configuración. Un proyecto representa una aplicación, biblioteca o componente de software, mientras que una tarea constituye una unidad de trabajo encargada de realizar una operación específica, como compilar archivos, ejecutar pruebas o generar paquetes distribuibles. Las tareas pueden establecer relaciones de dependencia entre ellas, permitiendo que Gradle construya un grafo dirigido acíclico que determina cuáles deben ejecutarse y en qué orden. Su ciclo de construcción comprende tres fases: inicialización, en la que identifica los proyectos participantes; configuración, en la que evalúa los archivos de construcción y determina las tareas necesarias; y ejecución, en la que realiza las operaciones seleccionadas respetando sus dependencias. Gradle también incorpora mecanismos de compilación incremental y almacenamiento en caché que permiten evitar la repetición de determinadas tareas cuando sus entradas y resultados no han cambiado. Por consiguiente, Gradle no constituye un lenguaje de programación ni un compilador específico, sino un sistema extensible que coordina herramientas, compiladores y procesos de construcción de software (Gradle, s. f.-c) (Gradle, s. f.-b).

La configuración de Gradle se realiza mediante archivos de construcción que utilizan un lenguaje específico de dominio (*Domain-Specific Language*, DSL), disponible en dos variantes: Groovy DSL y Kotlin DSL. En la primera se utilizan archivos con extensión .gradle, mientras que en la segunda se emplean archivos .gradle.kts, cuyo contenido está escrito en Kotlin y es compilado y ejecutado por Gradle como lógica de configuración. Estos archivos permiten aplicar complementos, declarar dependencias externas, configurar propiedades del proyecto y establecer tareas personalizadas. En proyectos compuestos por varios módulos, el archivo settings.gradle.kts identifica los módulos participantes y puede establecer los repositorios utilizados para resolver complementos y dependencias, mientras que los archivos build.gradle.kts definen la

configuración de construcción correspondiente a cada proyecto o módulo. Gradle dispone de un mecanismo de administración de dependencias que permite localizar, descargar, resolver y almacenar bibliotecas externas, considerando sus versiones y relaciones de dependencia. Además, proporciona Gradle Wrapper, un conjunto de archivos formado principalmente por gradlew, gradlew.bat, gradle-wrapper.jar y gradle-wrapper.properties, que permite ejecutar una versión determinada de Gradle sin exigir su instalación manual previa. Este mecanismo facilita que distintos desarrolladores y sistemas de integración continua utilicen la misma versión de la herramienta, siempre que se mantengan compatibles las demás dependencias y herramientas del proyecto (Gradle, s. f.-a) (Gradle, s. f.-d) (Gradle, s. f.-e).

En el desarrollo de aplicaciones Android, Gradle constituye la base del sistema de construcción utilizado por Android Studio, mientras que el complemento Android Gradle Plugin (AGP) proporciona la lógica especializada para procesar proyectos destinados a este sistema operativo. Al aplicar AGP, Gradle registra y configura las tareas necesarias para utilizar las herramientas del Android SDK, compilar código Java y Kotlin mediante los compiladores correspondientes, procesar recursos, integrar el archivo AndroidManifest.xml, generar código DEX y empaquetar los resultados en archivos APK o Android App Bundle (AAB). La configuración de cada módulo permite establecer parámetros como namespace, compileSdk, minSdk, targetSdk, las dependencias utilizadas y las características de las variantes de construcción. Estas variantes se forman a partir de tipos de construcción (*build types*), como debug y release, y, opcionalmente, de variantes de producto (*product flavors*), lo que permite generar diferentes versiones de una misma aplicación a partir de un proyecto compartido. En una aplicación híbrida basada en WebView, los archivos HTML, CSS y JavaScript incluidos en los recursos locales pueden incorporarse al paquete Android, mientras que las clases Kotlin se compilan mediante las herramientas correspondientes y se integran en el producto final. Gradle coordina estas operaciones, pero no transforma por sí mismo el código JavaScript en Kotlin ni sustituye a los compiladores o a las herramientas del Android SDK. Finalmente, el proceso de construcción puede iniciarse desde Android Studio o mediante comandos del Gradle Wrapper, como gradlew.bat :app:assembleDeb en Windows, para generar un APK de depuración del módulo app (Android Developers, 2026b) (Android Developers, 2026a).

Sync, Build, Run en Android Studio

Gradle, Sync y Build son dos operaciones diferentes utilizadas por Android Studio para preparar y construir aplicaciones Android. Gradle Sync es el proceso de sincronización mediante el cual Android Studio interpreta e importa la configuración de construcción del proyecto, identifica los módulos que lo componen, resuelve las dependencias necesarias y actualiza su modelo interno para reconocer correctamente las bibliotecas, las herramientas y las configuraciones declaradas. Esta operación permite que el entorno de desarrollo proporcione funcionalidades como el autocompletado, el análisis del código y la identificación de errores de configuración, sin que necesariamente se compile la aplicación ni se genere un archivo instalable. Por otra parte, Build corresponde al proceso de construcción del software, durante el cual se ejecutan las operaciones necesarias para transformar el código fuente y los recursos de la aplicación en resultados de compilación. Dependiendo de las tareas seleccionadas, este proceso puede incluir la compilación del código Java o Kotlin, el procesamiento de recursos, la generación de código ejecutable para Android y el empaquetado de archivos APK o Android App Bundle (AAB). Por consiguiente, Gradle Sync prepara y actualiza la configuración del proyecto dentro de Android Studio, mientras que Build ejecuta las operaciones necesarias para construir sus componentes y generar los productos correspondientes (Android Developers, 2026a) (Android Developers, 2026b).

Ambos procesos se relacionan con el Android Software Development Kit (SDK), que constituye el conjunto de plataformas, bibliotecas y herramientas proporcionadas por Google para desarrollar, compilar, depurar y probar aplicaciones destinadas al sistema operativo Android. El SDK incluye las definiciones de las API de Android correspondientes a diferentes niveles de versión, las herramientas de compilación (*SDK Build Tools*) y las herramientas de plataforma (*SDK Platform Tools*), entre las que se encuentra Android Debug Bridge (ADB), utilizado para comunicarse con dispositivos y emuladores. Durante Gradle Sync, Android Studio interpreta parámetros como compileSdk, que determina el nivel de API utilizado para compilar el proyecto, y verifica la configuración necesaria para utilizar las plataformas y herramientas correspondientes. Durante Build, las herramientas de construcción utilizan los componentes del SDK para procesar recursos, vincularlos con las API de Android y

generar los resultados necesarios para construir la aplicación. Por ejemplo, AAPT2 procesa los recursos Android y D8 transforma el código de bytes compatible con Java en archivos DEX, utilizados por el entorno de ejecución de Android. De esta manera, Gradle Sync permite reconocer y configurar la utilización del SDK dentro del proyecto, mientras que Build utiliza sus herramientas durante las operaciones de construcción. No obstante, el SDK no constituye por sí mismo el sistema encargado de coordinar todas estas operaciones (Android Developers, 2025) (Android Developers, 2026b).

La relación de Gradle, Sync y Build con Gradle se establece mediante el sistema de automatización de construcción utilizado por Android Studio. Gradle interpreta los archivos de configuración del proyecto, como settings.gradle.kts y build.gradle.kts, administra dependencias y organiza las operaciones de construcción mediante tareas relacionadas entre sí. Durante Gradle Sync, Android Studio utiliza la integración con Gradle para obtener información sobre la estructura, los módulos, las dependencias y las variantes del proyecto, incorporando estos datos a su entorno de desarrollo. Durante Build, Android Studio solicita a Gradle la ejecución de las tareas de construcción correspondientes. Para proyectos Android, estas tareas son proporcionadas principalmente por Android Gradle Plugin (AGP), un complemento que amplía las capacidades generales de Gradle y coordina herramientas del Android SDK y compiladores como los de Kotlin y Java. Gradle organiza su ciclo de construcción en tres fases: inicialización, que identifica los proyectos participantes; configuración, que evalúa los archivos de construcción y registra las tareas; y ejecución, que realiza las operaciones seleccionadas siguiendo sus relaciones de dependencia. Es importante distinguir que Gradle Sync no equivale simplemente a la fase de configuración del ciclo de Gradle, sino que constituye una operación específica de integración entre Android Studio y el sistema de construcción. Por tanto, Gradle administra y coordina las operaciones, AGP proporciona las tareas especializadas para Android y el SDK aporta las API y herramientas necesarias para construir la aplicación (Gradle, s. f.) (Android Developers, 2026c) (Android Developers, 2026a).

Criterio	Gradle Sync	Build	Run
Definición	Es la operación mediante la cual Android Studio sincroniza su modelo interno del proyecto con la configuración administrada por Gradle. Permite reconocer módulos, dependencias, variantes y propiedades de construcción (Android Developers, 2026b).	Es el proceso que transforma el código fuente y los recursos mediante tareas de construcción. Puede generar archivos intermedios o productos distribuibles, como APK y AAB, según las tareas seleccionadas (Android Developers, 2026f).	Es la operación que permite construir la aplicación cuando es necesario, instalarla o actualizarla en un dispositivo Android y ejecutar su componente de inicio (Android Developers, 2026a).
Objetivo principal	Actualizar la información de configuración que Android Studio utiliza para reconocer la estructura del proyecto y proporcionar herramientas de desarrollo (Android Developers, 2026b).	Verificar y ejecutar las operaciones de construcción necesarias para obtener los resultados solicitados, como un APK de depuración (Android Developers, 2026c).	Comprobar el comportamiento de la aplicación en ejecución mediante un dispositivo físico o un emulador Android (Android Developers, 2026a).
Archivos y elementos procesados	Interpreta la configuración declarada en settings.gradle.kts, build.gradle.kts y otros archivos del sistema de construcción. Identifica módulos, complementos, dependencias y variantes (Android Developers, 2026b).	Procesa archivos fuente Java y Kotlin, recursos Android, el manifiesto y las dependencias necesarias para construir la variante seleccionada (Android Developers, 2026f).	Utiliza los resultados de construcción correspondientes a la configuración seleccionada y los implementa en el dispositivo de destino (Android Developers, 2026a).
Relación con Gradle	Android Studio se comunica con Gradle para obtener el modelo del proyecto. La sincronización no equivale exclusivamente a la fase de configuración del ciclo de Gradle (Android Developers, 2026b).	Gradle organiza y ejecuta tareas según sus dependencias. Su ciclo comprende inicialización, configuración y ejecución, aunque puede reutilizar resultados anteriores (Gradle, s. f.).	Android Studio utiliza la configuración de ejecución y solicita las operaciones de construcción necesarias antes de instalar e iniciar la aplicación (Android Developers, 2026a).
Relación con Android Gradle Plugin	Permite que Android Studio conozca las configuraciones, módulos y variantes definidas mediante Android Gradle Plugin (AGP) (Android Developers, 2026d).	AGP registra y configura las tareas especializadas para compilar, procesar recursos, generar DEX y empaquetar aplicaciones Android (Android Developers, 2026f).	La variante seleccionada determina las tareas de construcción y el paquete que Android Studio prepara para implementarlo en el dispositivo (Android Developers, 2026d).
Relación con Android SDK	La sincronización permite identificar la configuración del SDK requerida por el proyecto, incluyendo parámetros como compileSdk y las dependencias del entorno Android (Android Developers, 2026e).	Las tareas de construcción utilizan las plataformas, API y herramientas del Android SDK necesarias para procesar recursos y generar los archivos de la aplicación (Android Developers, 2026f).	Utiliza las herramientas de implementación de Android Studio y del SDK para comunicarse con el dispositivo o emulador, instalar la aplicación e iniciarla (Android Developers, 2026a).
Generación de archivos APK y AAB	No tiene como finalidad generar paquetes instalables. Su resultado principal es la sincronización del modelo del proyecto con Android Studio (Android Developers, 2026b).	Puede generar APK o AAB cuando se ejecutan las tareas correspondientes. Por ejemplo, assembleDebug genera un APK de depuración (Android Developers, 2026c).	Puede producir o reutilizar resultados de construcción para implementar la aplicación. Su objetivo principal no es generar un paquete para distribución (Android Developers, 2026a).
Necesidad de un dispositivo Android	No requiere tener conectado un teléfono ni disponer de un emulador en ejecución. La sincronización se realiza sobre la configuración del proyecto (Android Developers, 2026b).	La generación convencional de un APK no requiere un dispositivo conectado. Sin embargo, determinadas tareas de pruebas instrumentadas sí pueden necesitarlo (Android Developers, 2026c).	Requiere seleccionar un dispositivo físico disponible o un dispositivo virtual configurado mediante Android Emulator para ejecutar la aplicación (Android Developers, 2026a).
Resultado satisfactorio	Android Studio dispone de un modelo actualizado y utilizable del proyecto, aunque todavía pueden existir errores en el código fuente (Android Developers, 2026b).	Las tareas solicitadas se completan correctamente y quedan disponibles sus resultados. Esto no demuestra que la aplicación esté libre de errores de ejecución (Android Developers, 2026f).	La aplicación se implementa y se inicia en el dispositivo de destino. Posteriormente puede evaluarse su comportamiento mediante pruebas y herramientas de depuración (Android Developers, 2026a).

Cuadro 1: Comparación de las características técnicas, objetivos, herramientas y resultados de Gradle Sync, Build y Run en Android Studio.

Criterio	Gradle Sync	Build	Run
Momento recomendado de utilización	Al abrir un proyecto, modificar dependencias, actualizar complementos, incorporar módulos o cambiar archivos de configuración de Gradle (Android Developers, 2026b).	Cuando se necesita comprobar la construcción del proyecto, generar un APK, preparar una variante o obtener resultados de compilación (Android Developers, 2026c).	Cuando se necesita utilizar la aplicación en un teléfono o emulador para comprobar visualmente su interfaz y comportamiento (Android Developers, 2026a).
Cambios en el código fuente	Los cambios ordinarios en archivos Kotlin o Java no requieren ejecutar manualmente una sincronización, salvo que también cambie la configuración de construcción (Android Developers, 2026b).	Permite procesar los cambios del código fuente mediante las tareas de compilación correspondientes, según sus dependencias y entradas (Gradle, s. f.).	Permite comprobar los cambios en el dispositivo tras realizar las operaciones de construcción e implementación necesarias (Android Developers, 2026a).
Cambios en dependencias y bibliotecas	Permite que Android Studio actualice el reconocimiento de las nuevas dependencias y sus configuraciones declaradas en Gradle (Android Developers, 2026b).	Resuelve y utiliza las dependencias necesarias para las tareas de construcción. Puede fallar si una biblioteca requerida no se encuentra disponible (Android Developers, 2026f).	Requiere que la aplicación pueda construirse con las dependencias necesarias antes de implementarla correctamente (Android Developers, 2026a).
Errores característicos	Pueden presentarse errores de configuración, complementos, repositorios, resolución de dependencias o incompatibilidades del entorno de construcción (Android Developers, 2026b).	Pueden detectarse errores de compilación, recursos, manifiesto, resolución de dependencias y otras tareas necesarias para construir la aplicación (Android Developers, 2026f).	Pueden producirse errores al seleccionar el dispositivo, instalar el paquete, iniciar la actividad o ejecutar las funcionalidades de la aplicación (Android Developers, 2026a).
Herramientas de diagnóstico	La pestaña Sync de la ventana Build muestra información sobre la sincronización y sus errores (Android Developers, 2026a).	La pestaña Build Output permite examinar las tareas y los mensajes de error. Build Analyzer ayuda a estudiar el rendimiento de la construcción (Android Developers, 2026a).	Las herramientas de ejecución, depuración y Logcat permiten inspeccionar el funcionamiento de la aplicación en el dispositivo (Android Developers, 2026a).
Variantes Debug y Release	Permite actualizar el modelo de las variantes definidas mediante tipos de construcción y variantes de producto (Android Developers, 2026d).	Puede construir variantes debug y release mediante las tareas apropiadas. Sus opciones de firma y optimización pueden diferir (Android Developers, 2026e).	Implementa la variante seleccionada en la configuración de ejecución, siempre que sea compatible con el dispositivo y pueda instalarse (Android Developers, 2026d).
Automatización y línea de comandos	Es una operación de integración de Android Studio con Gradle. Una construcción ejecutada directamente desde la terminal no necesita una sincronización previa del IDE (Android Developers, 2026b).	Puede ejecutarse desde la terminal mediante Gradle Wrapper. En Windows, el comando gradlew.bat :app:assembleDebug permite construir el APK de depuración del módulo app (Android Developers, 2026c).	El comando gradlew.bat :app:installDebug permite construir e instalar la variante de depuración. Para iniciarla también debe ejecutarse su actividad, operación que Android Studio realiza mediante Run (Android Developers, 2026c) (Android Developers, 2026a).
Aplicación al proyecto de calculadora	Permite reconocer el módulo app, las configuraciones Gradle y las dependencias necesarias para el proyecto Android desarrollado con Kotlin (Android Developers, 2026b).	Permite compilar las clases Kotlin y empaquetar los recursos de la aplicación, incluidos los archivos web locales y el modelo GLB, cuando están incorporados a los recursos del proyecto (Android Developers, 2026f).	Permite instalar y abrir la calculadora en Android para comprobar la representación de WebView, las interacciones y las funcionalidades implementadas en Kotlin (Android Developers, 2026a).
Relación con la distribución	No genera por sí misma una aplicación preparada para entregar a los usuarios (Android Developers, 2026b).	Permite generar archivos APK o AAB destinados a pruebas o distribución, según la tarea seleccionada, la variante y su firma (Android Developers, 2026c).	Se utiliza principalmente para ejecutar y evaluar la aplicación durante el desarrollo, no como procedimiento de publicación en Google Play (Android Developers, 2026a).

Cuadro 2: Comparación de las aplicaciones prácticas, errores, herramientas de diagnóstico y procedimientos de Gradle Sync, Build y Run durante el desarrollo de una aplicación Android.

2 Métodos para Desarrollar una Aplicación móvil

El Método I utiliza una arquitectura híbrida en la que la clase WebView, perteneciente al paquete android.webkit del framework de Android, permite cargar y representar una interfaz gráfica desarrollada con HTML, CSS y JavaScript dentro de una aplicación Android. HTML define la estructura de los elementos de la interfaz, CSS establece sus propiedades visuales y JavaScript implementa la lógica de interacción y las operaciones de la aplicación. Kotlin permite instanciar, configurar y controlar WebView, además de acceder a las funcionalidades del sistema operativo mediante las API de Android. La interacción entre JavaScript y Kotlin puede implementarse mediante el método addJavascriptInterface(), que registra una instancia de una clase Kotlin en WebView y la expone al entorno JavaScript mediante un identificador. De esta manera, JavaScript puede invocar los métodos públicos de dicha clase anotados con @JavascriptInterface, permitiendo ejecutar operaciones implementadas en Kotlin, incluidas aquellas que utilizan las API de Android. En sentido inverso, Kotlin puede utilizar el método evaluateJavascript() para ejecutar código JavaScript de forma asíncrona en el contexto de la página cargada. Esta arquitectura permite reutilizar interfaces desarrolladas con tecnologías web e incorporar funcionalidades nativas de Android mediante mecanismos de interoperabilidad definidos por la API de WebView (Android Developers, s. f.-a) (Android Developers, s. f.-b).

El Método II utiliza una arquitectura de desarrollo multiplataforma basada en React Native y TypeScript. TypeScript permite definir los componentes de la interfaz gráfica, sus propiedades, el estado de la aplicación y la lógica de interacción mediante un sistema de tipos estáticos. Durante el proceso de construcción, Babel transforma el código TypeScript en JavaScript para su ejecución en el entorno de React Native.

A diferencia de WebView, React Native no requiere representar la interfaz mediante documentos HTML, sino que utiliza componentes de React, como View, Text y TextInput, cuya representación y actualización se administran mediante el sistema de renderizado Fabric. En Android, este sistema genera y actualiza

las vistas correspondientes del framework, como android.view.ViewGroup y android.widget.TextView (React Native, s. f.-e) (React Native, s. f.-c).

El acceso a las funcionalidades del dispositivo puede realizarse mediante las API proporcionadas por React Native, bibliotecas de terceros o módulos nativos personalizados. En este último caso, la arquitectura de React Native permite implementar módulos mediante Kotlin para utilizar las API del sistema operativo Android. Los Turbo Native Modules permiten definir en TypeScript las interfaces, los métodos y los tipos de datos disponibles para el código JavaScript. A partir de estas especificaciones, Codegen genera las interfaces necesarias para integrar la implementación nativa con el entorno de ejecución de React Native (React Native, s. f.-a) (React Native, s. f.-d).

La comunicación con servicios externos puede implementarse mediante la API Fetch, que permite realizar solicitudes HTTP o HTTPS de manera asíncrona, enviar información al servidor y procesar sus respuestas. Por consiguiente, tanto el Método I como el Método II pueden consumir los mismos endpoints de una API alojada en un servidor, intercambiando información mediante formatos como JSON, independientemente de las tecnologías utilizadas para implementar sus interfaces gráficas (React Native, s. f.-b).

La fluidez del Método I depende del procesamiento y renderizado del contenido web: cuando estas tareas exceden el tiempo disponible por fotograma, pueden producirse interrupciones en las animaciones y los desplazamientos. React Native permite ejecutar determinadas animaciones y desplazamientos en el hilo nativo de la interfaz, lo que puede favorecer la fluidez.

Método I: Utilizar HTML, CSS, JavaScript con AppView para visualización y Kotlin para acceder a las funciones nativas del dispositivo.

En Android, WebView permite sustituir pantallas y controles de interfaz nativos por contenido web integrado dentro de la aplicación. Así, la interfaz se presenta y gestiona mediante HTML, CSS y JavaScript, mientras que Kotlin aloja y configura el WebView y conecta el contenido web con las capacidades del dispositivo cuando es necesario. Por tanto, WebView reemplaza la implementación nativa de la interfaz, pero el acceso a funciones como la cámara o los sensores sigue dependiendo de las API de Android, mediante mecanismos web compatibles o un puente de comunicación con Kotlin.

- HTML: Definir la estructura y el contenido: qué elementos existen en la pantalla (MDN Web Docs, s. f.).

- CSS: Definir la presentación visual: cómo se ven y distribuyen los elementos del HTML (MDN Web Docs, s. f.).

- JavaScript: Implementar el comportamiento y la lógica: qué sucede cuando el usuario interactúa y cómo se procesan los datos (MDN Web Docs, s. f.).

- Kotlin: Implementar la integración nativa con Android: comunicación con el sistema operativo, gestión de pantallas y acceso a funciones del dispositivo cuando la aplicación las necesita (Android Developers, s. f.-e).

Método II: Utilizar React Native con TypeScript para desarrollar interfaces con componentes nativos así como acceder a las funciones del dispositivo móvil Android

En Android, React Native permite construir la interfaz de la aplicación desde TypeScript utilizando componentes que se representan mediante vistas nativas de Android. De esta manera, sustituye la escritura directa de las pantallas en Kotlin por una implementación gestionada desde React Native. El acceso a las funciones del dispositivo se realiza mediante las API y los módulos nativos disponibles; cuando se requiere una capacidad que estos no proporcionan, es posible implementar un módulo en Kotlin y conectarlo con el código TypeScript.

- React Native: Desarrollar la interfaz de la aplicación mediante componentes nativos y acceder a funciones del dispositivo a través de las API y los módulos disponibles.

- TypeScript: Implementar la lógica y el comportamiento de la aplicación mediante JavaScript con tipado estático, lo que permite detectar errores de tipos durante el desarrollo.

Tabla comparativa entre los Métodos I y II

Criterio	Método I: HTML, CSS y JavaScript + WebView y Kotlin	Método II: React Native + TypeScript
Visualización de la interfaz	La interfaz desarrollada con HTML, CSS y JavaScript se representa dentro de la clase WebView del framework de Android mediante el motor de renderizado basado en Chromium (Chrome for Developers, 2024).	La interfaz se define mediante componentes React y TypeScript. El renderizador de React Native transforma estos componentes en vistas de Android y administra sus actualizaciones (React Native, 2024) (React Native, 2026n).
Acceso a funciones del dispositivo	Kotlin permite utilizar las API de Android. El método addJavascriptInterface() registra objetos cuyos métodos públicos anotados con @JavascriptInterface pueden invocarse desde JavaScript (Android Developers, 2026h).	React Native proporciona API y módulos para utilizar funcionalidades del dispositivo. Cuando se requieren operaciones adicionales de Android, pueden implementarse Turbo Native Modules en Kotlin y exponer sus interfaces al código TypeScript (React Native, 2026l).
Reutilización de contenido web	Permite incorporar archivos HTML, CSS y JavaScript existentes dentro de WebView, tanto desde recursos empaquetados como desde contenido remoto (Android Developers, 2026a) (Android Developers, 2026d).	Los componentes visuales HTML no se representan directamente mediante el renderizador de React Native. Para construir la interfaz mediante vistas nativas es necesario adaptarlos a componentes React Native. La lógica JavaScript compatible puede reutilizarse (React Native, 2024) (React Native, 2026h).
Animaciones, listas y transiciones	Se implementan mediante tecnologías web como CSS y JavaScript. Su rendimiento depende de las operaciones de estilo, composición, representación y ejecución de instrucciones JavaScript (Lewis, 2023).	Pueden implementarse mediante componentes como Animated y FlatList. Las animaciones compatibles pueden utilizar un controlador nativo y las listas pueden limitar los elementos montados simultáneamente (React Native, 2026b) (React Native, 2026f).
Acceso a datos mediante API	JavaScript puede realizar solicitudes HTTP o HTTPS mediante fetch() para consultar y modificar información de una API. Las solicitudes están sujetas a las políticas de seguridad del entorno web, incluidas las restricciones de origen (Android Developers, 2026d).	React Native proporciona la API Fetch para realizar solicitudes HTTP o HTTPS. Puede utilizar los mismos endpoints del servidor que el Método I, siempre que se respeten los requisitos de autenticación, protocolo y datos (React Native, 2026e).
Fluidez y calidad visual	Las operaciones de JavaScript, cálculo de estilos, distribución de elementos y representación pueden afectar al tiempo necesario para generar cada fotograma. La fluidez depende de la implementación y de los recursos disponibles en el dispositivo (Lewis, 2023).	React Native distingue el procesamiento de JavaScript de las actualizaciones de la interfaz. Determinadas animaciones pueden ejecutarse mediante un controlador nativo, aunque las tareas costosas y las actualizaciones pueden producir pérdida de fotogramas (React Native, 2026b) (React Native, 2026g).
Consumo de CPU y memoria	El procesamiento del DOM, JavaScript y los recursos gráficos utiliza memoria del entorno WebView, incluida memoria nativa y, según la configuración, procesos de renderizado separados. La administración del ciclo de vida es importante para evitar fugas de memoria. No puede inferirse un consumo inferior al de React Native (Android Developers, 2026e).	El consumo depende de los componentes, imágenes, operaciones JavaScript y vistas montadas. FlatList permite ajustar la cantidad de elementos renderizados y la ventana de montaje para administrar la memoria. No puede inferirse un consumo inferior al de WebView (React Native, 2026f).
Consumo de energía	Las operaciones de red, el procesamiento y las actualizaciones de contenido pueden contribuir al consumo energético. Optimizar la frecuencia de las solicitudes y la transferencia de datos puede reducir el trabajo de los subsistemas de conectividad (Android Developers, 2026f).	El consumo energético también depende de la actividad de red, el procesamiento y las actualizaciones visuales. La arquitectura no garantiza por sí misma un menor consumo respecto de WebView; dicha diferencia requiere mediciones comparables (Android Developers, 2026f) (React Native, 2026g).
Ventaja según la arquitectura	Facilita reutilizar una interfaz web existente y ejecutarla dentro de una aplicación Android, con la posibilidad de incorporar operaciones implementadas mediante Kotlin (Android Developers, 2026a).	Facilita desarrollar interfaces basadas en componentes React que se representan mediante vistas del sistema operativo y compartir lógica entre plataformas compatibles (React Native, 2024) (React Native, 2026h).

Cuadro 3: Comparación técnica de la arquitectura, la visualización y los factores de rendimiento de los métodos propuestos para desarrollar aplicaciones Android.

Criterio	Método I: HTML, CSS y JavaScript + WebView y Kotlin	Método II: React Native + TypeScript
Motor de ejecución de JavaScript	El código JavaScript se ejecuta en el entorno web de Android WebView, basado en Chromium. Este entorno también interpreta HTML y CSS para representar la interfaz (Chrome for Developers, 2024).	React Native utiliza Hermes como motor de ejecución de JavaScript de manera predeterminada. Hermes es independiente del motor de renderizado web y está optimizado para React Native (React Native, 2026m).
Depuración del código	Permite inspeccionar el DOM, los estilos CSS, la ejecución de JavaScript y las solicitudes de red mediante Chrome DevTools. La implementación Kotlin puede depurarse con las herramientas de desarrollo de Android (Android Developers, 2026b).	React Native DevTools permite inspeccionar componentes React, ejecutar depuración de JavaScript y examinar información de rendimiento. La implementación Kotlin requiere herramientas de depuración del entorno nativo de Android (React Native, 2026i).
Pruebas automatizadas	Espresso-Web permite localizar elementos HTML y comprobar interacciones dentro de WebView mediante pruebas instrumentadas de Android. La lógica JavaScript también puede evaluarse mediante pruebas independientes (Android Developers, 2026c).	Permite desarrollar pruebas unitarias, pruebas de componentes y pruebas de extremo a extremo. La documentación contempla herramientas como Jest y bibliotecas especializadas de pruebas (React Native, 2026k).
Funcionamiento sin conexión	Puede cargar archivos HTML, CSS, JavaScript e imágenes empaquetados con la aplicación mediante WebViewAssetLoader. Las funciones dependientes de servicios remotos requieren datos almacenados localmente o mecanismos de sincronización (Android Developers, 2026d).	Puede utilizar código y recursos estáticos incluidos en la aplicación, como imágenes y otros archivos empaquetados. Para consultar datos originalmente remotos sin conexión se necesita implementar almacenamiento local y, cuando corresponda, sincronización (React Native, 2026d).
Navegación y botón de retroceso	WebView mantiene un historial de navegación consultable con canGoBack(). El método goBack() permite regresar a una página anterior. La aplicación puede integrar ese comportamiento con la navegación de Android (Android Developers, 2026h).	La API BackHandler permite responder a acciones de retroceso del sistema Android. La navegación entre pantallas debe gestionarse mediante la lógica de la aplicación o una biblioteca de navegación compatible (React Native, 2026c).
Persistencia de datos locales	WebView permite utilizar almacenamiento web como localStorage cuando está habilitada la API de almacenamiento DOM. También pueden implementarse mecanismos nativos de persistencia mediante Kotlin (Android Developers, 2026g).	Puede utilizar bibliotecas como Async Storage para datos no sensibles. Este almacenamiento no proporciona cifrado por defecto. Para información sensible deben emplearse mecanismos de almacenamiento seguro (React Native, 2026j).
Portabilidad a otros sistemas operativos	Los archivos HTML, CSS y JavaScript pueden reutilizarse en entornos web compatibles. Sin embargo, la integración específica de Android mediante Kotlin debe adaptarse para otras plataformas, como iOS, donde existe la clase WKWebView (Apple, s. f.).	Permite compartir componentes y lógica JavaScript o TypeScript entre Android e iOS. Las diferencias entre las API y los módulos nativos pueden requerir implementaciones específicas para cada sistema operativo (React Native, 2026h).
Accesibilidad	Puede implementar accesibilidad mediante HTML semántico, etiquetas descriptivas, estructuras de navegación y atributos ARIA, siguiendo las recomendaciones del W3C para desarrollar contenido web accesible (World Wide Web Consortium, 2026).	Proporciona propiedades como accessible, accessibilityLabel y accessibilityRole para describir los componentes a los servicios de accesibilidad, incluido TalkBack en Android (React Native, 2026a).
Seguridad de la integración	El método addJavascriptInterface() expone métodos de objetos registrados al contenido JavaScript de WebView. Es necesario restringir esta capacidad a contenido confiable, ya que el mecanismo no permite identificar de forma segura el origen de cada marco que invoca los métodos (Android Developers, 2024).	Requiere administrar de forma segura las credenciales, las dependencias y el almacenamiento de información sensible. Los datos confidenciales no deben guardarse en almacenamiento persistente sin cifrado, como Async Storage (React Native, 2026j).

Cuadro 4: Comparación de los mecanismos de ejecución, depuración, pruebas, almacenamiento, seguridad y portabilidad de los métodos propuestos para el desarrollo de aplicaciones Android.

Sección de Preguntas

¿Por qué se utiliza la librería React.js de JavaScript?

React es una librería de JavaScript que permite desarrollar interfaces de usuario de manera estructurada, modular y eficiente. A diferencia del desarrollo con JavaScript puro, donde gran parte de la manipulación y actualización de los elementos HTML debe realizarse manualmente, React permite organizar la interfaz de una aplicación en componentes, piezas que se definen una vez y pueden emplearse en distintas partes de la pantalla. Por ejemplo, en una calculadora, un mismo componente de botón puede servir para todos los números, cambiando el texto que muestra y la acción que realiza al pulsarlo. Las características comunes de esos botones pueden modificarse desde un único lugar, lo que facilita mantener un diseño uniforme. (React, s. f.)

Facilitando su mantenimiento, escalabilidad y desarrollo. Por esta razón, se recomienda utilizar React para aplicaciones que incorporen múltiples pantallas, autenticación de usuarios, mapas interactivos, conexión con bases de datos y comunicación con servicios externos, como Microsoft Azure.

Además, React puede combinarse con HTML, CSS y JavaScript para integrar la aplicación web en un proyecto de Android Studio y generar un archivo APK. De esta manera, es posible desarrollar una aplicación móvil aprovechando tecnologías web, sin necesidad de implementar toda la interfaz mediante Kotlin o Java. (React, s. f.; Android Developers, s. f.-a)

¿Por qué utilizar Kotlin y no Java?

Java y Kotlin se compilan al mismo tipo de bytecode, que posteriormente es interpretado o traducido a código máquina por la máquina virtual de Java (JVM), permitiendo que el procesador ejecute las instrucciones del programa. (JetBrains, s. f.-c; Oracle, 2025)

Fue diseñado, entre otras cosas, para reducir la cantidad de código necesario respecto de Java, manteniendo la compatibilidad con su ecosistema. Especialmente su máquina virtual (JVM) y sus bibliotecas. Ambos tienen compiladores distintos, pero pueden generar el mismo tipo de bytecode. (JetBrains, s. f.-c)
. (JetBrains, s. f.-c)

Kotlin puede llamar directamente a funciones y clases escritas en Java, sin necesidad de convertirlas a Kotlin. Esto se conoce como interoperabilidad entre ambos lenguajes y es posible porque ambos pueden utilizar la misma máquina virtual (JVM). (JetBrains, s. f.-a)

JVM ejecuta un programa Kotlin como si hubiese sido compilado desde Java. Esto es posible porque ambos lenguajes pueden generar el mismo tipo de bytecode. Kotlin no se transforma en código fuente Java. (JetBrains, s. f.-c)

La JVM (Java Virtual Machine), o Máquina Virtual de Java, es un programa que permite ejecutar aplicaciones compiladas en bytecode, como las desarrolladas en Java o Kotlin. Su función principal es transformar o interpretar las instrucciones del bytecode para que el procesador (CPU) pueda ejecutarlas. (Oracle, 2025)

La CPU (Central Processing Unit), o Unidad Central de Procesamiento, es el procesador principal de un computador. Es un componente físico (hardware) encargado de ejecutar las instrucciones de los programas. Por ejemplo, los procesadores Intel Core, AMD Ryzen y Apple M son CPU.

El código escrito en Kotlin se compila primero a código de bytes para la máquina virtual de Java y después se convierte al formato DEX. En el dispositivo, el entorno de ejecución de Android (ART) puede compilar ese código DEX a lenguaje de máquina, es decir, a instrucciones específicas que la CPU del

dispositivo Android ejecuta directamente.

Criterio	Java	Kotlin
Declaración de variables e inferencia de tipos	Las variables se declaran indicando su tipo o utilizando var para inferir el tipo de determinadas variables locales, funcionalidad incorporada desde Java 10. La palabra final impide reasignar una variable. La inferencia mediante var no se aplica a los campos de las clases (Oracle, 2026a).	Utiliza val para declarar referencias que no pueden reasignarse y var para referencias reasignables. Permite inferir los tipos de variables locales y propiedades a partir de sus expresiones de inicialización. Una referencia val no garantiza que el objeto referenciado sea inmutable (Kotlin, s. f.-a).
Tratamiento de valores nulos	Los tipos de referencia pueden contener null. El sistema de tipos del lenguaje no distingue de manera general entre referencias anulables y no anulables. Pueden utilizarse comprobaciones explícitas y anotaciones adicionales para documentar la nulabilidad (Kotlin, s. f.-e).	Distingue entre tipos no anulables, como String, y anulables, como String?. Incorpora operadores como ?. y ?: para administrar estos valores. La protección se aplica principalmente durante la compilación, pero no elimina todas las posibles excepciones por referencias nulas, especialmente al interoperar con Java (Kotlin, s. f.-j).
Clases para representar datos	Permite declarar clases convencionales y clases record, incorporadas de forma definitiva en Java 16. Los registros generan constructores, métodos de acceso, equals(), hashCode() y toString(). Sus campos son finales, aunque los objetos referenciados pueden ser mutables (Oracle, 2026b).	Proporciona data class, que genera automáticamente equals(), hashCode(), toString(), componentN() y copy() a partir de las propiedades del constructor primario. Estas propiedades pueden declararse mediante val o var; copy() realiza una copia superficial (Kotlin, s. f.-g).
Propiedades y métodos de acceso	Los atributos de una clase se representan mediante campos. Es habitual controlar su lectura y modificación mediante métodos como getNombre() y setNombre(). Los accesores pueden implementarse explícitamente para incorporar validaciones y otras operaciones (Kotlin, s. f.-c).	Incorpora propiedades como construcción propia del lenguaje, utilizando val y var. El compilador proporciona accesores y permite personalizar get() y set(). En la JVM, las propiedades pueden exponerse a Java mediante métodos accesores compatibles (Kotlin, s. f.-k).
Parámetros y llamadas a funciones	Los métodos reciben argumentos según su posición. Para ofrecer distintas combinaciones de parámetros es habitual utilizar sobrecarga de métodos. El lenguaje no proporciona argumentos nombrados ni valores predeterminados de parámetros como características generales de sus métodos (Kotlin, s. f.-e).	Permite declarar funciones con fun, asignar valores predeterminados a los parámetros y especificar argumentos mediante sus nombres. Estas características permiten omitir ciertos argumentos y reducir la necesidad de crear múltiples sobrecargas de una misma función (Kotlin, s. f.-i).
Tratamiento de excepciones	Distingue excepciones comprobadas y no comprobadas. Las excepciones comprobadas deben capturarse o declararse mediante throws, de acuerdo con las reglas del lenguaje. Esta obligación se verifica durante la compilación (Oracle, s. f.).	No exige declarar ni capturar excepciones comprobadas. Permite utilizar try, catch, finally y throw, pero el compilador no impone la misma obligación que Java. Al exponer funciones Kotlin a Java puede emplearse @Throws para declarar excepciones en las firmas interoperables (Kotlin, s. f.-e) (Kotlin, s. f.-c).
Extensión de funcionalidades	Permite añadir operaciones relacionadas con tipos existentes mediante clases auxiliares, métodos estáticos, composición u otras estructuras. No incorpora funciones de extensión con la misma sintaxis de receptor que Kotlin (Kotlin, s. f.-h).	Admite funciones y propiedades de extensión que pueden invocarse con la sintaxis de un miembro del tipo receptor. Estas extensiones no modifican la clase original ni agregan realmente miembros a ella. Habitualmente, su resolución es estática (Kotlin, s. f.-h).
Comprobación de tipos	Admite el operador instanceof y reconocimiento de patrones, que permite comprobar el tipo de un objeto y obtener una variable tipada. Las versiones modernas también admiten patrones en determinadas expresiones switch (Oracle, 2026a).	Utiliza is y !is para comprobar tipos. Cuando el compilador puede demostrar que una variable pertenece a un tipo específico, realiza una conversión inteligente o smart cast, sin necesidad de escribir una conversión explícita (Kotlin, s. f.-e).
Colecciones y mutabilidad	Ofrece interfaces como List, Set y Map. La posibilidad de modificar una colección depende de su implementación o de los envoltorios utilizados, sin que la interfaz List distinga por sí sola las colecciones modificables de las no modificables (Kotlin, s. f.-d).	Distingue interfaces de solo lectura, como List, Set y Map, de interfaces mutables como MutableList. Esta distinción restringe las operaciones disponibles durante la compilación, aunque una interfaz de solo lectura no garantiza inmutabilidad profunda del objeto (Kotlin, s. f.-d).

Cuadro 5: Comparación de las características sintácticas, del sistema de tipos y de los mecanismos de programación de Java y Kotlin.

Criterio	Java	Kotlin
Compatibilidad y soporte oficial de Android	Es compatible con el SDK de Android, Android Studio y numerosas bibliotecas AndroidX. Google mantiene la posibilidad de implementar aplicaciones y acceder a las API del sistema mediante Java, aunque el desarrollo de herramientas y documentación nuevas prioriza Kotlin (Android Developers, 2026d).	Es un lenguaje plenamente compatible con Android y el enfoque prioritario de Google para nuevas herramientas, ejemplos y API. Android Studio ofrece integración para crear, depurar y mantener proyectos Kotlin, sin impedir su combinación con archivos Java (Android Developers, 2026d) (Android Developers, 2026g).
Acceso a las API de Android	Permite utilizar las clases del framework de Android para implementar actividades, servicios, almacenamiento, sensores, conectividad y otras funcionalidades disponibles mediante el SDK (Android Developers, 2026d).	Permite utilizar las mismas API del framework, incluidas aquellas implementadas originalmente en Java. La interoperabilidad con Java permite invocar sus clases y métodos desde Kotlin, considerando las diferencias de tipos y nulabilidad (Android Developers, 2026g) (Kotlin, s. f.-b).
Interfaces gráficas mediante Views y XML	Permite implementar actividades en Java y cargar interfaces declaradas mediante recursos XML, utilizando métodos como setContentView(). También permite crear y modificar objetos derivados de View mediante código (Android Developers, 2026h).	Permite implementar actividades en Kotlin y utilizar los mismos recursos XML y componentes del sistema tradicional de vistas de Android. También admite construir y modificar la jerarquía de vistas mediante código Kotlin (Android Developers, 2026h).
Desarrollo de interfaces con Jetpack Compose	No permite declarar directamente funciones @Composable mediante la sintaxis Java, ya que Jetpack Compose está construido alrededor de Kotlin. Sin embargo, una aplicación Java puede integrar componentes Compose implementados en módulos o clases Kotlin (Android Developers, 2026f).	Es el lenguaje utilizado para implementar directamente interfaces con Jetpack Compose, mediante funciones @Composable, lambdas, parámetros con valores predeterminados y administración declarativa del estado (Android Developers, 2026f).
Concurrencia y tareas asíncronas	Permite ejecutar operaciones concurrentes mediante hilos, ejecutores y otras API de concurrencia. En Android pueden utilizarse mecanismos como Thread y ExecutorService para realizar trabajo fuera del hilo principal (Android Developers, 2026e).	Permite utilizar corrutinas mediante kotlinx.coroutines, funciones suspend y ámbitos CoroutineScope. Las corrutinas permiten suspender operaciones sin bloquear necesariamente un hilo y facilitan la administración estructurada de tareas asíncronas (Kotlin, s. f.-f) (Android Developers, 2026e).
Interoperabilidad entre lenguajes	Puede instanciar clases Kotlin e invocar sus métodos compilados para la JVM. Algunas características de Kotlin requieren adaptaciones, como @JvmStatic, @JvmField o @JvmOverloads, según la interfaz que se desee exponer (Kotlin, s. f.-c).	Puede utilizar clases, interfaces y métodos Java directamente. Para tipos Java sin información de nulabilidad, Kotlin puede utilizar tipos de plataforma cuya seguridad respecto de null requiere atención adicional (Kotlin, s. f.-b).
Bibliotecas y extensiones AndroidX	Puede utilizar las API públicas compatibles de AndroidX y del framework de Android. Sin embargo, las extensiones que dependen de construcciones específicas del lenguaje Kotlin no se invocan desde Java con su sintaxis idiomática (Android Developers, 2026d).	Además de las API generales de AndroidX, puede aprovechar extensiones KTX, que utilizan funciones de extensión, propiedades, lambdas, argumentos nombrados y corrutinas para ofrecer interfaces de programación adaptadas a Kotlin (Android Developers, 2026c).
Compilación y compatibilidad de versiones	El código fuente Java se compila a archivos de clases que las herramientas de Android procesan para generar código DEX. La disponibilidad de determinadas características del lenguaje y de sus bibliotecas depende de la configuración del proyecto y del soporte de desugaring (Android Developers, 2025).	El compilador Kotlin para JVM genera archivos de clases que también se procesan para obtener código DEX. La combinación de versiones de Kotlin, Android Gradle Plugin, D8 y R8 debe respetar los requisitos de compatibilidad documentados para cada versión (Android Developers, 2026a).
Reutilización de código entre plataformas	Java permite reutilizar código en entornos compatibles con sus API y con su modelo de ejecución. Sin embargo, disponer de código Java para Android no implica que pueda ejecutarse directamente como código nativo de iOS (Android Developers, 2026d).	Kotlin Multiplatform permite organizar módulos compartidos y reutilizar lógica, modelos de datos y operaciones entre Android, iOS y otras plataformas. Las funcionalidades específicas del sistema operativo pueden requerir implementaciones diferenciadas (Kotlin, s. f.-l).
Rendimiento y consumo de recursos	El rendimiento depende de los algoritmos, las asignaciones de memoria, las estructuras de datos, las bibliotecas y la ejecución sobre Android. No puede establecerse una ventaja general de Java frente a Kotlin sin mediciones comparables (Android Developers, 2026g).	Kotlin no introduce por sí mismo una ventaja o desventaja universal de rendimiento. Determinadas operaciones pueden generar asignaciones o comprobaciones adicionales. El consumo de CPU, memoria y energía debe evaluarse para implementaciones y dispositivos específicos (Android Developers, 2026g).

Cuadro 6: Comparación de Java y Kotlin en el desarrollo de aplicaciones Android, considerando compatibilidad, interfaces gráficas, concurrencia, interoperabilidad, compilación y rendimiento.

¿Cuando conviene usar JavaScript sin React.js para desarrollar la aplicación?

JavaScript puro puede resultar conveniente para desarrollar interfaces y elementos interactivos muy personalizados cuando no se requiere el sistema de componentes y administración del estado de React.js. Sin embargo, React.js no limita la creación de componentes personalizados y permite integrar funcionalidades desarrolladas previamente con JavaScript convencional. Esta integración puede realizarse mediante referencias a elementos del DOM y efectos que permiten coordinar bibliotecas o componentes externos con el ciclo de vida de React (React, s. f.-b) (React, s. f.-c).

Asimismo, JavaScript puro puede preferirse para implementar funcionalidades específicas que no necesitan el sistema de renderizado y actualización de componentes de React.js. Esta elección depende de los requisitos y de la complejidad de la aplicación, y no implica que React.js imponga restricciones generales a la personalización de las interfaces. De hecho, la documentación oficial contempla la incorporación gradual de React en proyectos existentes, sin necesidad de reemplazar toda su implementación original (React, s. f.-a).

¿Qué diferencia hay entre React.js, React Native y React DOM?

React.js es una librería de JavaScript para construir interfaces mediante componentes. En aplicaciones web se utiliza junto con React DOM para representar esos componentes como elementos del documento HTML, cuya apariencia se define mediante CSS.

React Native es un framework que utiliza el modelo de componentes de React.js para desarrollar aplicaciones que, en Android, presentan su interfaz mediante componentes nativos de la plataforma.

Por tanto, ambos comparten el modelo de componentes para organizar la interfaz y permiten utilizar JavaScript o TypeScript, pero difieren en cómo la presentan: React.js, junto con React DOM, utiliza elementos HTML para la web, mientras que React Native utiliza componentes nativos de la plataforma móvil.

Esto permite compartir parte de la lógica, aunque la interfaz generalmente requiere adaptaciones entre dispositivos.

React.js se utiliza principalmente para desarrollar interfaces web, mientras que React Native permite desarrollar aplicaciones móviles utilizando componentes nativos de Android e iOS. (React, s. f.; React Native, s. f.-c)

React Native es un framework de desarrollo de aplicaciones móviles, es decir, un conjunto de bibliotecas, herramientas y componentes de software que permite crear aplicaciones para Android e iOS utilizando JavaScript o TypeScript. (React Native, s. f.-c, s. f.-e)

Específicamente es un conjunto de archivos de código y bibliotecas que instalas en tu proyecto. Incluye componentes desarrollados en JavaScript, C++, Java/Kotlin y Objective-C/Swift. (React Native, s. f.-b)

No garantiza completamente que una aplicación funcione correctamente en Android e iOS, pero sí está diseñado para desarrollar aplicaciones para ambos sistemas operativos utilizando una gran parte del mismo código. (React Native, s. f.-c)

Se instala en tu proyecto y después puedes importar sus componentes y funciones, igual que haces con una biblioteca de JavaScript. (React Native, s. f.-c)

Permite comunicar JavaScript con componentes desarrollados en diferentes lenguajes de programación (React Native, s. f.-b).

¿Qué lenguajes de programación se utilizan para React Native y conjuntamente con él?

1cm

- **JavaScript**: es un lenguaje de programación estandarizado mediante ECMAScript que permite implementar la lógica de las aplicaciones desarrolladas con React Native. Se utiliza para definir componentes React mediante archivos .js o .jsx, administrar sus propiedades y estados, implementar funciones, procesar eventos de interacción, realizar operaciones matemáticas y ejecutar solicitudes HTTP a servicios externos. React Native utiliza JavaScript para describir la estructura lógica de la interfaz y determinar sus actualizaciones, mientras que su sistema de renderizado se encarga de crear y modificar las vistas correspondientes del sistema operativo. A diferencia del desarrollo web convencional, esta ejecución no requiere un documento HTML ni el Document Object Model (DOM) del navegador. El código JavaScript se ejecuta mediante un motor compatible con ECMAScript, siendo Hermes el motor utilizado de manera predeterminada por React Native. Además, JavaScript puede invocar las funcionalidades expuestas por módulos nativos implementados en otros lenguajes (Ecma International, 2026) (React Native, s. f.-c).

- **TypeScript**: es un lenguaje de programación desarrollado por Microsoft que extiende JavaScript mediante un sistema de tipos estáticos. Permite declarar explícitamente los tipos de variables, parámetros, valores de retorno, propiedades y estructuras de datos, además de definir interfaces y tipos genéricos. En React Native se utiliza para programar componentes mediante archivos .ts y .tsx, describir las propiedades que reciben, definir la estructura de los estados y comprobar la compatibilidad de los datos utilizados por las funciones. También permite especificar las interfaces de los Turbo Native Modules, utilizadas por Codegen para generar parte del código necesario para integrar la lógica JavaScript con implementaciones nativas. Durante la construcción de la aplicación, Babel transforma el código TypeScript en JavaScript, eliminando las anotaciones de tipos. Por consiguiente, TypeScript no utiliza un motor de ejecución independiente y sus verificaciones de tipos se realizan durante el desarrollo, sin constituir comprobaciones automáticas de tipos en tiempo de ejecución. React Native utiliza TypeScript de manera predeterminada en los proyectos nuevos, aunque mantiene compatibilidad con JavaScript (Microsoft, s. f.) (React Nati-

ve, 2026).

- **Kotlin / Java**: son lenguajes de programación utilizados para implementar funcionalidades específicas de Android mediante las API proporcionadas por su framework y sus bibliotecas. Kotlin es un lenguaje de tipado estático, compatible con el ecosistema Java, que permite desarrollar clases, funciones y módulos capaces de utilizar servicios del sistema operativo, como almacenamiento local, sensores, cámara, conectividad y administración de actividades. Java también permite implementar estas funcionalidades y continúa siendo compatible con las API y bibliotecas de Android. En React Native, ambos lenguajes pueden utilizarse para desarrollar Turbo Native Modules personalizados cuando las funcionalidades disponibles desde JavaScript o TypeScript no satisfacen los requisitos de la aplicación. Estos módulos implementan operaciones nativas y las exponen al entorno JavaScript mediante interfaces definidas por React Native. En la Nueva Arquitectura, Codegen permite generar contratos e infraestructura de interoperabilidad a partir de especificaciones tipadas. Kotlin y Java pueden coexistir dentro del mismo proyecto Android, sin que sea necesario utilizar ambos para implementar cada funcionalidad (Kotlin, s. f.) (Oracle, 2026) (React Native, s. f.-d).

- **Swift / Objective-C**: son lenguajes utilizados para implementar funcionalidades específicas de los sistemas operativos de Apple, particularmente iOS. Swift es un lenguaje de programación de propósito general con tipado estático y mecanismos de seguridad de tipos y memoria, que permite desarrollar clases, estructuras, funciones y módulos capaces de utilizar las API de Apple, como las proporcionadas por Foundation, UIKit y otros frameworks del sistema. Objective-C es un lenguaje orientado a objetos basado en C que proporciona un sistema de envío dinámico de mensajes y mantiene compatibilidad con numerosas bibliotecas y componentes del ecosistema de Apple. En React Native, estos lenguajes permiten implementar módulos nativos que proporcionan funcionalidades de iOS y que pueden ser utilizados desde JavaScript o TypeScript. La Nueva Arquitectura admite implementaciones de módulos en Objective-C++ y también permite incorporar lógica desarrollada en Swift. En determinadas integraciones, Swift requiere una capa de adaptación escrita en Objective-C++ para interactuar con las interfaces de C++ utilizadas internamente por React Native. Por tanto, Swift y Objective-C cumplen funciones de integración con iOS equivalentes, en términos arquitectónicos,

a las desempeñadas por Kotlin y Java en Android, aunque utilizan API y mecanismos específicos de Apple (Swift, s. f.) (Apple, s. f.) (React Native, s. f.-b).

- **C++**: es un lenguaje de programación compilado, estandarizado mediante ISO/IEC 14882, que permite implementar algoritmos, estructuras de datos, componentes de sistemas y bibliotecas que requieren control sobre los recursos computacionales. En React Native se utiliza principalmente en componentes internos de su Nueva Arquitectura, entre ellos partes del sistema de renderizado Fabric y de los mecanismos de interoperabilidad entre JavaScript y las implementaciones nativas. Fabric comparte una implementación central desarrollada en C++ entre distintas plataformas, mientras que JavaScript Interface (JSI) permite establecer referencias e invocar operaciones entre el entorno JavaScript y objetos implementados en C++. Además, React Native admite Turbo Native Modules implementados en C++, lo que permite compartir determinadas operaciones y algoritmos entre Android e iOS sin duplicar toda su implementación. Sin embargo, el código C++ que necesita utilizar API específicas de un sistema operativo puede requerir adaptaciones adicionales para cada plataforma. Su utilización resulta especialmente pertinente para desarrollar bibliotecas compartidas, procesar información mediante algoritmos especializados o integrar componentes nativos existentes, aunque no es necesaria para implementar la lógica habitual de todas las aplicaciones React Native (ISO C++ Foundation, s. f.) (React Native, 2024) (React Native, s. f.-a).

¿Que diferencia hay entre WebView y React Native ?

WebView es un componente de Android basado en Chromium que permite mostrar contenido web dentro de una aplicación. En este proyecto, procesa los archivos HTML y CSS y ejecuta el código JavaScript de la calculadora, incluidos en la propia aplicación. Por tanto, no necesita abrir un navegador externo ni descargar una página de Internet.

No necesita instalar Chromium como navegador. Android WebView incluye los componentes basados en Chromium necesarios para mostrar el contenido web dentro de tu calculadora.

Viene normalmente preinstalado como un componente del sistema llamado Android System WebView, que las aplicaciones utilizan para mostrar contenido web.

La principal diferencia entre WebView y React Native es que WebView permite desarrollar interfaces utilizando HTML, CSS y JavaScript, las cuales se ejecutan dentro de un motor de navegador integrado en la aplicación, mientras que React Native utiliza JavaScript o TypeScript para construir interfaces mediante componentes nativos de Android e iOS, sin depender de un navegador para renderizarlas.

En WebView, se puede utilizar Kotlin para acceder a las funcionalidades nativas del dispositivo y comunicarlas con JavaScript, mientras que React Native proporciona APIs y módulos para acceder a estas funcionalidades. WebView ofrece mayor flexibilidad para reutilizar tecnologías web, mientras que React Native suele ofrecer una experiencia más cercana a una aplicación nativa, especialmente en interfaces móviles complejas.

React Native utiliza componentes nativos y dispone de optimizaciones para el renderizado, las animaciones y la comunicación entre JavaScript y el sistema operativo (React Native, s. f.-a, s. f.-b, s. f.-d).

WebView ejecuta su interfaz dentro de Chromium, un motor web muy optimizado, pero que también requiere memoria y recursos adicionales para funcionar (Android Developers, s. f.-d).

Descripción General de la Aplicación Móvil

La aplicación móvil, identificada en la interfaz como **Casio fx-2200**, presenta una recreación funcional de una calculadora científica de finales de la década de 1970. El tutorial App I combina una ficha histórica, una representación tridimensional interactiva y una calculadora bidimensional que permite realizar operaciones aritméticas, científicas y estadísticas. El proyecto incluye el código fuente de Android y un archivo de instalación APK correspondiente a la versión 2.0.2.

El usuario puede recorrer tres secciones mediante las pestañas **Historia**, **Modelo 3D** y **Cálculo**. La primera presenta los antecedentes del instrumento y permite consultar sus fuentes documentales. La segunda muestra un modelo en formato .glb, que puede girarse y acercarse mediante gestos, además de contar con un control para restablecer la cámara. La tercera presenta el teclado de la calculadora, sus indicadores y un manual de operación dividido en cinco capítulos.

La interfaz utiliza HTML, CSS y JavaScript, integrados en Android mediante una WebView, componente que permite incorporar contenido web en una aplicación Android (Android Developers, 2026e). Su presentación emplea un fondo de papel beige, texto negro y la tipografía local Special Elite. El modelo tridimensional se visualiza mediante Three.js, mientras que el motor de cálculo está desarrollado en Kotlin. Este lenguaje también permite acceder al portapapeles, compartir resultados, reproducir sonidos y generar respuesta táctil mediante las funcionalidades nativas del dispositivo.

La aplicación instalada realiza los cálculos y carga sus recursos sin conexión a Internet. El HTML, las hojas de estilo, los archivos JavaScript, la tipografía, el sonido y el modelo 3D se incorporan al APK. Los ajustes de modo, sonido y respuesta táctil se guardan localmente mediante SharedPreferences, una API para almacenar valores en pares clave-valor (Android Developers, 2026b). Los registros de cálculo, la memoria y los datos estadísticos se conservan durante la navegación interna, pero se reinician cuando se recrea el proceso de Android.

Compilación e instalación de la aplicación móvil

Código fuente y repositorio GitHub
Interfaz HTML, CSS y JavaScript
Código Kotlin, modelo GLB y configuración Gradle

↓

Android Studio: preparación del entorno
Apertura de la carpeta raíz del proyecto
Selección de JDK 17 y disponibilidad de SDK Platform 36
Disponibilidad de SDK Build-Tools 35.0.0

↓

Sync: sincronización del proyecto con Gradle
Lectura de settings.gradle.kts y de los archivos build.gradle.kts
Reconocimiento de módulos, variantes y opciones de construcción
Resolución de complementos y dependencias
Descarga de los componentes necesarios que falten

Resultado: Android Studio dispone del modelo del proyecto para reconocer su estructura y sus dependencias; no genera el APK

↓

Build: construcción del APK de depuración
Gradle ejecuta la tarea :app:assembleDebug
Android Gradle Plugin coordina las tareas de Android

Código de la aplicación	**Manifiesto y recursos**
Compilación de las clases Kotlin	Procesamiento del manifiesto y de res
Motor de cálculo e integración con Android	Incorporación de HTML, CSS y JavaScript
Conversión del código compilado al formato DEX	Three.js, modelo GLB, tipografía y sonido

Empaquetado del código y los recursos, alineación y firma con la clave de depuración

↓

Resultado de Build: archivo APK de depuración

Paquete firmado e instalable para realizar pruebas

app/build/outputs/apk/debug/app-debug.apk

↓

Dispositivo Android: instalación y ejecución

Instalación del APK e inicio de la aplicación en el teléfono

Android 8.0 o posterior, según minSdk = 26

Fundamento técnico: Android Developers (2026a, 2026f) y GitHub (s. f.).

Elemento	Qué es y para qué sirve	Función en la construcción y prueba de esta aplicación
Gradle 8.13	Sistema que automatiza la construcción de un proyecto. Administra las dependencias, ejecuta las tareas y coordina el orden en que deben realizarse (Android Developers, 2026f).	Lee la configuración del proyecto y coordina la compilación de Kotlin, el procesamiento de recursos y la generación del paquete instalable. La tarea assembleDebug inicia la construcción del APK de depuración (Android Developers, 2026a). El Gradle Wrapper del proyecto establece el uso de la versión 8.13.
Android Gradle Plugin 8.12.2	Complemento que incorpora a Gradle las tareas y opciones específicas para construir aplicaciones Android. Interpreta configuraciones como compileSdk, minSdk y targetSdk (Android Developers, 2026f).	Coordina el procesamiento del manifiesto y los recursos, la transformación del código compilado al formato utilizado por Android y el empaquetado del APK. El proyecto lo aplica mediante el complemento com.android.application.
Kotlin 2.1.20	Lenguaje de programación utilizado para implementar la lógica de la aplicación. Su complemento de Gradle integra el compilador de Kotlin en el proceso de construcción (JetBrains, 2026).	Compila las clases MainActivity, AppView, CalculatorEngine y ButtonSound. Estas implementan el inicio de la app, la WebView, los cálculos y las funciones nativas del teléfono. El código compilado se incorpora posteriormente al paquete Android.
JDK 17	Java Development Kit: conjunto que proporciona el entorno Java y herramientas necesarias para ejecutar los procesos de construcción (Android Developers, 2026g).	Proporciona la máquina virtual Java donde se ejecutan Gradle y las herramientas de compilación. Se utiliza en el computador durante la construcción. La compatibilidad de Java y el destino JVM de Kotlin también están configurados en 17 dentro del proyecto.
Android SDK Build-Tools 35.0.0	Conjunto de herramientas del SDK destinadas a la construcción de aplicaciones Android, incluidas utilidades para preparar, alinear y firmar paquetes APK (Android Developers, 2026h).	Sus herramientas participan en las tareas de construcción y preparación del APK. La versión 35.0.0 es la predeterminada de Android Gradle Plugin 8.12 (Android Developers, 2026l); el proyecto no establece otra mediante buildToolsVersion.
APK	Android Package: archivo instalable que reúne el código compilado, el manifiesto y los recursos de una aplicación Android (Android Developers, 2026f).	Es el resultado del proceso de construcción. Contiene el código de la calculadora, la interfaz HTML/CSS/JavaScript, las bibliotecas de Three.js, el modelo GLB, la tipografía y el sonido. El APK de depuración generado normalmente se encuentra en app/build/outputs/apk/debug/app-debug.apk.
Android SDK Platform-Tools	Paquete de herramientas que permite interactuar con dispositivos Android. Incluye ADB, utilizado durante el desarrollo y las pruebas (Android Developers, 2026j).	Interviene en la instalación y las pruebas posteriores a la compilación. Proporciona las herramientas mediante las cuales Android Studio detecta el teléfono conectado y se comunica con él para instalar y ejecutar la aplicación.
ADB	Android Debug Bridge: herramienta incluida en SDK Platform-Tools que permite establecer comunicación entre el computador y Android (Android Developers, 2026d).	Permite enviar e instalar el APK en el teléfono, iniciar la aplicación y consultar registros para investigar errores. En este caso, se utiliza mediante la conexión USB con la depuración habilitada y autorizada en el dispositivo.
Android 8.0	Versión del sistema operativo Android correspondiente al nivel de API 26 (Android Developers, 2026i). En este proyecto representa la versión mínima de instalación.	El valor minSdk = 26 se incorpora a la configuración de la aplicación para establecer qué dispositivos pueden instalarla. La compilación utiliza SDK Platform 36, mientras que Android 8.0 define el límite inferior de ejecución previsto por el proyecto.

Cuadro 7: Función de las herramientas, el paquete resultante y el requisito mínimo de Android en la compilación, instalación y prueba de la aplicación Casio fx-2200.

Elemento	Información relevante	Aplicación en la construcción de Casio fx-2200
Android SDK Platform 36	Contiene las APIs de Android utilizadas como referencia durante la compilación. Es un componente diferente de Build-Tools y Platform-Tools (Android Developers, 2026g, 2026i).	El valor compileSdk = 36 indica que el código se compila utilizando las APIs de Android 16. Esto permite comprobar las referencias a las clases y funciones de Android. La versión mínima de instalación se establece por separado mediante minSdk = 26.
Configuración Gradle del proyecto	El archivo build.gradle.kts situado en la raíz declara las versiones de los complementos de construcción (Android Developers, 2026f).	Establece Android Gradle Plugin 8.12.2 y Kotlin 2.1.20. Estas declaraciones permiten resolver los complementos que después utiliza el módulo de la aplicación.
Configuración Gradle de la aplicación	El archivo app/build.gradle.kts define cómo se construye el módulo app, sus dependencias y sus opciones de Android (Android Developers, 2026f).	Declara los niveles de SDK, el identificador com.retrolab.casiofx2200, la versión 2.0.2 y el código de versión 5. También establece la compatibilidad Java y el destino JVM de Kotlin en 17.
Configuración de módulos y repositorios	El archivo settings.gradle.kts identifica los módulos del proyecto y los repositorios donde se buscan complementos y dependencias (Android Developers, 2026f).	Incluye el módulo :app. Configura repositorios como Google, Maven Central y Gradle Plugin Portal, según el tipo de componente que debe resolverse durante la construcción.
Gradle Wrapper	Conjunto formado por los scripts gradlew, gradlew.bat y sus archivos de soporte. Permite utilizar la versión de Gradle indicada por el proyecto (Android Developers, 2026a, 2026f).	El archivo gradle-wrapper.properties establece la distribución Gradle 8.13. El Wrapper obtiene esa distribución si falta y la utiliza para ejecutar las tareas de construcción.
Manifiesto de Android	El archivo AndroidManifest.xml declara componentes de la aplicación, permisos y características que Android necesita conocer para ejecutarla (Android Developers, 2026e).	Identifica MainActivity como actividad de inicio, establece el nombre visible y configura opciones como la aceleración gráfica. Su contenido se procesa e incorpora al APK.
Carpeta assets	Almacena archivos que la aplicación recupera mediante sus rutas locales. Su contenido se incorpora al APK (Android Developers, 2026k).	Incluye HTML, CSS, JavaScript, las bibliotecas de Three.js, la tipografía y el modelo GLB. La WebView carga estos archivos durante la ejecución. El JavaScript se ejecuta en la WebView; el motor de cálculo Kotlin se compila mediante su complemento.
Recursos de Android	La carpeta res contiene recursos organizados por tipo. El sistema de construcción los procesa y genera identificadores para utilizarlos desde el código (Android Developers, 2026b).	Incluye el icono, los estilos, la configuración de extracción de datos y el sonido button_click.mp3. Kotlin accede al sonido mediante R.raw.button_click.
Dependencia AndroidX WebKit	Biblioteca que proporciona utilidades de integración y compatibilidad para trabajar con WebView. Las dependencias se resuelven mediante Gradle (Android Developers, 2026c, 2026f).	El proyecto declara androidx.webkit:webkit:1.14.0. Proporciona clases como WebViewAssetLoader y WebViewCompat, utilizadas para cargar recursos locales y comunicar JavaScript con Kotlin (Android Developers, 2026c, 2026k).
Firma del APK de depuración	La construcción de tipo debug utiliza una clave de depuración para firmar el APK generado. La firma forma parte de la preparación del paquete instalable (Android Developers, 2026a).	La tarea assembleDebug genera un APK firmado para realizar pruebas en el teléfono. Esta firma corresponde al entorno de desarrollo; una distribución de publicación requiere su propia configuración de firma.

Cuadro 8: Archivos de configuración, recursos y componentes que intervienen en la construcción de la aplicación Casio fx-2200.

Etapa 1. Código fuente y GitHub: almacenamiento del proyecto. El proyecto reúne los archivos de la interfaz, las clases Kotlin, el modelo tridimensional y los recursos necesarios para ejecutar la aplicación. GitHub Desktop permite gestionar el repositorio local y registrar los cambios del código fuente; su publicación en GitHub permite conservar una copia remota y compartir el proyecto (GitHub, s. f.).

Etapa 2. Android Studio: preparación del entorno.

Esta etapa consiste en disponer de las herramientas y dependencias necesarias antes de compilar la aplicación. Al abrir el proyecto, Android Studio interpreta settings.gradle.kts, build.gradle.kts y app/build.gradle.kts para reconocer sus módulos, repositorios, complementos y opciones de construcción.

El Gradle Wrapper establece el uso de Gradle 8.13, mientras que los archivos del proyecto declaran Android Gradle Plugin 8.12.2 y Kotlin 2.1.20. Durante la sincronización se resuelven los complementos y las dependencias, incluida AndroidX WebKit 1.14.0, y se descargan los componentes que todavía no están disponibles localmente. Estas versiones se leen desde el proyecto y no necesitan introducirse manualmente en Settings (Android Developers, 2026f).

La preparación del computador requiere seleccionar JDK 17 como entorno Java para ejecutar Gradle y disponer de Android SDK Platform 36, que proporciona las APIs utilizadas como referencia al compilar, y Android SDK Build-Tools 35.0.0, que aporta utilidades para la construcción del paquete Android (Android Developers, 2026g, 2026h, 2026l). También se necesita Android SDK Platform-Tools para disponer de ADB, la herramienta de comunicación con el teléfono que se utilizará posteriormente durante las pruebas (Android Developers, 2026d, 2026j). Los valores compileSdk = 36 y targetSdk = 36 permanecen establecidos en el proyecto. Por su parte, minSdk = 26 identifica Android 8.0 como requisito mínimo del teléfono; no representa otro componente que deba instalarse en el computador para preparar la compilación.

La preparación también comprende comprobar que el proyecto contenga AndroidManifest.xml, donde se declaran los componentes y características de la apli-

cación; la carpeta assets, con la interfaz web, las bibliotecas JavaScript y el modelo tridimensional; y la carpeta res, con los recursos de Android, como el icono, los estilos y el sonido (Android Developers, 2026b, 2026e, 2026k). Además, se reconoce la configuración de construcción debug, destinada a producir posteriormente un APK de pruebas firmado automáticamente con una clave de depuración. El APK constituye el paquete instalable que se obtendrá al construir el proyecto; en esta etapa se prepara su configuración, mientras que su generación corresponde a las etapas siguientes (Android Developers, 2026a).

Etapa 3. Gradle: compilación y empaquetado. Gradle organiza la construcción de la aplicación y utiliza Android Gradle Plugin para ejecutar las tareas específicas de Android. El complemento de Kotlin compila el código que implementa el motor de cálculo y la integración con las funciones del dispositivo. Su uso ya está declarado en los archivos Gradle, por lo que no es necesario instalar Kotlin manualmente. Durante la construcción también se procesan los recursos de Android y se incorporan los archivos de la carpeta assets. El HTML, CSS y JavaScript se incluyen como recursos locales que la WebView interpreta durante la ejecución. El modelo GLB y las bibliotecas de Three.js también forman parte del paquete. El sistema de construcción reúne el código y los recursos para generar la aplicación Android (Android Developers, 2026f).

Etapa 4. APK: generación del archivo de instalación. La tarea assembleDebug permite generar un APK de depuración en app/build/outputs/apk/debug/app-debug.apk (Android Developers, 2026a). El archivo ZIP entregado incluye además el paquete Casio-fx2200-debug.apk. La documentación de la revisión 2.0.2 señala que este APK se actualizó mediante el reempaquetado de los recursos web y del modelo GLB sobre código nativo previamente validado; no registra una nueva compilación Gradle para esa revisión.

Etapa 5. Android: instalación y ejecución en el teléfono. Para probar la aplicación desde Android Studio, se conecta un teléfono compatible al computador, se activa la depuración USB en las opciones de desarrollador del dispositivo y se acepta la autorización de conexión. Después, se selecciona el teléfono en Android Studio y se pulsa **Run** para compilar, instalar y ejecutar la aplicación. También puede instalarse el APK de depuración generado previamente (Android Developers, 2026a). Al abrir la aplicación, Android inicia la actividad principal y presenta la interfaz mediante una WebView. El funcionamiento cotidiano utiliza el código y los recursos incluidos en el paquete. Las referencias históricas se abren en el navegador externo cuando el usuario selecciona sus enlaces.

Funcionamiento de la aplicación móvil

Android: MainActivity y AppView
Inicio de la aplicación y creación de la WebView
Carga de los recursos locales incluidos en el APK

↓

Interfaz HTML, CSS y JavaScript
Pestañas Historia, Modelo 3D y Cálculo
Teclado, manual, ajustes e indicadores

En Modelo 3D: Three.js visualiza el archivo GLB local.
En Cálculo: cada tecla envía una acción al código Kotlin.

↓

Puente de mensajes JavaScript-Kotlin
Intercambio de solicitudes y respuestas en formato JSON
Mensajes aceptados desde el documento local principal

↓

Motor Kotlin: CalculatorEngine
Procesamiento de teclas y operaciones
Actualización del valor, memoria, modo y datos SD

↓

Respuesta a la interfaz
Kotlin devuelve el estado de la calculadora
JavaScript actualiza el visor y los indicadores

↓

Funciones complementarias gestionadas por Kotlin
Copiar, compartir, sonido y respuesta táctil
SharedPreferences conserva los ajustes de modo, sonido y tacto

Fundamento técnico: Android Developers (2026b, 2026d, 2026g) y Three.js (s. f.-a, s. f.-b).

Cuando el usuario abre la aplicación, Android ejecuta MainActivity. Esta actividad crea una instancia de AppView, una clase Kotlin basada en WebView, y la establece como contenido de la pantalla. Mediante WebViewAssetLoader, la aplicación carga el documento index.html desde sus recursos locales. Aunque se utiliza una dirección con el dominio appassets.androidplatform.net, la respuesta se obtiene de los archivos incluidos en el APK. Este mecanismo permite cargar HTML, CSS y JavaScript empaquetados con la aplicación (Android Developers, 2026g).

La WebView interpreta el HTML y el CSS, y ejecuta el JavaScript encargado de la navegación y la presentación. Al iniciar, la interfaz muestra la sección Historia. El usuario puede pasar al modelo tridimensional o a la calculadora mediante las pestañas superiores. El manual de operación se abre desde Cálculo y presenta cinco capítulos: operaciones, funciones científicas, fracción, memoria y estadística SD.

Al acceder por primera vez a Modelo 3D, JavaScript carga el módulo scene.js. Este utiliza Three.js para crear la escena, la cámara y la iluminación; GLTFLoader carga el archivo casio_fx2200.glb y OrbitControls permite controlar la cámara para observar el modelo desde distintos ángulos y acercarlo. Estas funciones de carga y control de cámara están documentadas por Three.js (s. f.-a, s. f.-b). El encuadre se adapta al tamaño del modelo y del área disponible. La animación se pausa al abandonar esta sección o cuando el documento deja de estar visible.

En Cálculo, cada pulsación genera una solicitud JavaScript que contiene un identificador, una acción y, cuando corresponde, la tecla seleccionada. La solicitud se convierte a JSON y se envía mediante NativeApp.postMessage. El receptor registrado en Kotlin utiliza WebViewCompat.addWebMessageListener, que permite establecer los orígenes autorizados para los mensajes (Android Developers, 2026d). La aplicación comprueba además que el mensaje provenga del marco principal y del origen local permitido antes de procesarlo. Las pulsaciones de cálculo se envían en secuencia para conservar el orden de las operaciones.

La clase CalculatorEngine, desarrollada en Kotlin, mantiene el estado numé-

rico y procesa las teclas. JavaScript recoge las pulsaciones y actualiza la presentación, mientras que Kotlin ejecuta las operaciones. Ambos componentes funcionan localmente en el teléfono. Por ejemplo, en modo DEG, JavaScript envía las pulsaciones correspondientes al número 30 y a la función seno; Kotlin calcula el resultado y devuelve el valor 0.5 para mostrarlo en el visor. La tecla MODE recorre RAD, DEG, GRA y SD. En SD, determinadas teclas permiten obtener la suma de datos, la suma de cuadrados, la cantidad de observaciones, la media y las desviaciones estándar muestral y poblacional.

El motor comprueba condiciones como la división por cero, los argumentos fuera del dominio real y la falta de datos para una operación estadística. Si una operación no es válida, actualiza el estado de error y su mensaje explicativo. La tecla C permite borrar la entrada y corregirla conservando la operación pendiente, mientras que AC borra el cálculo. La memoria independiente y los datos SD cuentan con controles de borrado separados.

Después de procesar una acción, Kotlin devuelve una respuesta JSON con el estado actualizado. Esta respuesta incluye el valor del visor, el modo, la memoria, la operación pendiente, los indicadores y los avisos. JavaScript relaciona la respuesta con su solicitud y actualiza los elementos de la interfaz sin volver a cargar todo el documento.

Las funciones del dispositivo también se gestionan desde Kotlin. El resultado puede copiarse mediante el portapapeles o compartirse mediante una intención de Android; el mecanismo de compartir texto entre aplicaciones se basa en ACTION_SEND (Android Developers, 2026h). Los sonidos de pulsación se reproducen con SoundPool, una clase que permite cargar y reproducir muestras breves de audio desde recursos locales (Android Developers, 2026c). La respuesta táctil se solicita mediante las APIs del sistema. Las preferencias permiten activar o desactivar ambos efectos y recuperar esos ajustes al volver a iniciar la aplicación.

La persistencia distingue entre los ajustes y el estado de trabajo. SharedPreferences conserva el modo seleccionado, el sonido y la respuesta táctil mediante pares clave-valor (Android Developers, 2026b). Los valores del cálculo, la memoria M y las observaciones SD permanecen en el motor Kotlin duran-

te la navegación entre las pantallas, pero no se guardan de manera permanente. Por ello se reinician al recrearse el proceso de Android.

Finalmente, la WebView restringe la navegación y el acceso a recursos. Se deshabilitan el acceso directo a archivos y contenido, y el contenido mixto; además, el documento establece una política de seguridad de contenido. La documentación de carga local explica las restricciones de acceso a archivos y de contenido mixto (Android Developers, 2026g). El manifiesto no solicita el permiso INTERNET. Los enlaces históricos pertenecen a una lista permitida y Kotlin los abre en el navegador externo. De esta forma, la interfaz, el cálculo y la visualización tridimensional funcionan con los componentes locales de la aplicación.

Estructura del Repositorio de GitHub

```
App-Tutorial-I-main/                      # Carpeta del repositorio
|-- README.md                             # Presentación del tutorial
|-- Casio-fx2200-debug.apk                # Aplicación Android instalable
|
‘-- CasioFX2200-proyecto/                  # Proyecto de Android Studio
    |-- .gitignore                        # Exclusiones Git
    |-- README.md                         # Arquitectura y uso
    |-- FUENTES.md                        # Referencias históricas
    |-- VERIFICACION.md                   # Verificaciones documentadas
    |-- THIRD_PARTY_NOTICES.md            # Avisos de terceros
    |-- build.gradle.kts                  # Plugins Android y Kotlin
    |-- settings.gradle.kts               # Módulos y repositorios
    |-- gradle.properties                 # Opciones de compilación
    |-- gradlew                           # Gradle para Linux y macOS
    |-- gradlew.bat                       # Gradle para Windows
    |
    |-- gradle/
    |   ‘-- wrapper/
    |       |-- gradle-wrapper.jar        # Ejecutor del Gradle Wrapper
    |       ‘-- gradle-wrapper.properties  # Versión y descarga de Gradle
    |
    ‘-- app/                              # Módulo de la aplicación
        |-- build.gradle.kts              # SDK, versión y dependencias
        ‘-- src/
            |-- main/
            |   |-- AndroidManifest.xml   # Declaración de la aplicación
            |   |
            |   |-- assets/               # Interfaz y archivos locales
            |   |   |-- index.html        # Historia, modelo y calculadora
            |   |   |-- styles.css        # Estilos y diseño adaptable
            |   |   |-- app.js            # Interfaz y mensajes a Kotlin
            |   |   |-- scene.js          # Escena y cámara del visor 3D
            |   |   |-- models/
            |   |   |   ‘-- casio_fx2200.glb # Modelo de la calculadora
            |   |   ‘-- vendor/
            |   |       |-- font/
            |   |       |   |-- SpecialElite-Regular.ttf # Tipografía
            |   |       |   ‘-- LICENSE.txt # Licencia de la fuente
            |   |       ‘-- three/
            |   |           |-- three.module.js # Biblioteca Three.js
            |   |           |-- GLTFLoader.js # Carga del modelo GLB
            |   |           |-- OrbitControls.js # Giro y acercamiento
            |   |           |-- BufferGeometryUtils.js # Geometrías
            |   |           ‘-- LICENSE.txt # Licencia de Three.js
            |   |
            |   |-- kotlin/com/retrolab/casiofx2200/
            |   |   |-- MainActivity.kt   # Inicio y ciclo de vida
            |   |   |-- AppView.kt        # WebView y funciones Android
            |   |   |-- CalculatorEngine.kt # Motor de cálculo científico
            |   |   ‘-- ButtonSound.kt    # Reproducción de sonidos
            |   |
            |   ‘-- res/                  # Recursos nativos de Android
            |       |-- drawable/
            |       |   ‘-- ic_calculator.xml # Icono de la aplicación
            |       |-- raw/
            |       |   ‘-- button_click.mp3 # Sonido de botones
            |       |-- values/
            |       |   ‘-- styles.xml     # Tema de la aplicación
            |       |-- values-v27/
            |       |   ‘-- styles.xml     # Tema para API 27 y posteriores
            |       ‘-- xml/
            |           ‘-- data_extraction_rules.xml # Reglas de respaldo
            |
            ‘-- test/kotlin/com/retrolab/casiofx2200/
                |-- CalculatorEngineTest.kt # Pruebas del motor de cálculo
                ‘-- PreviewHarness.kt     # Adaptador de verificación JVM
```

En el proyecto, HTML, CSS y JavaScript se utilizan para desarrollar la interfaz interactiva de la calculadora Casio fx-2200. HTML organiza las pantallas de historia, modelo tridimensional y cálculo; CSS define su presentación visual; y JavaScript administra la navegación, los botones y la actualización de la información mostrada. Además, Three.js permite visualizar de forma interactiva el modelo tridimensional incluido en los recursos de la aplicación.

La interfaz se ejecuta dentro de un componente WebView integrado en la aplicación Android. Cuando el usuario pulsa una tecla de la calculadora, JavaScript envía un mensaje al código Kotlin mediante el puente de comunicación configurado en AppView.kt. Kotlin procesa la solicitud utilizando CalculatorEngine.kt y devuelve el estado actualizado para que JavaScript muestre el resultado en pantalla. Las funciones del dispositivo, como copiar al portapapeles, compartir resultados, reproducir sonidos y generar respuesta táctil, también se ejecutan desde Kotlin.

Durante la construcción, Android Studio utiliza Gradle y las herramientas de Android para compilar el código Kotlin y empaquetarlo junto con los archivos HTML, CSS, JavaScript, las bibliotecas locales, la tipografía, el sonido y el modelo tridimensional. El resultado es un archivo APK que puede instalarse en un dispositivo Android. Una vez instalada, la aplicación realiza los cálculos y muestra el modelo 3D sin conexión a Internet, ya que los recursos necesarios se encuentran incluidos en el propio paquete. Las referencias históricas externas se abren en el navegador del dispositivo cuando el usuario decide consultarlas.

Las siguientes clases pueden utilizarse desde Kotlin gracias a su compatibilidad con Java, sin necesidad de escribir archivos fuente Java propios. Permiten crear la pantalla, mostrar la interfaz web, organizar sus elementos, configurar colores y examinar

- Activity (android.app.Activity): Proporciona la pantalla nativa que aloja la calculadora y los métodos para gestionar su ciclo de vida, por ejemplo, cuando se crea, pasa a segundo plano o se destruye (Android Developers, s. f.-e).

- WebView (android.webkit.WebView): Es el componente que muestra la interfaz HTML, aplica los estilos CSS y ejecuta el código JavaScript de la calculadora dentro de la pantalla Android (Android Developers, s. f.-a).

- FrameLayout (android.widget.FrameLayout): Es un contenedor de elementos visuales. En la calculadora, contiene el WebView y permite configurar el espacio que ocupa dentro de la pantalla (Android Developers, s. f.-e).

- Color (android.graphics.Color): Proporciona constantes y métodos para representar y manipular colores. Se utiliza para establecer el color de fondo del contenedor nativo y del WebView (Android Developers, s. f.-e).

- Uri (android.net.Uri): Permite representar y analizar identificadores de recursos, como las direcciones de los archivos locales. En la calculadora, se utiliza para examinar las direcciones que el WebView intenta cargar (Android Developers, s. f.-e).

Estas clases forman parte de Android y se utilizan desde nuestro código Kotlin. No hemos escrito sus archivos Java: son componentes proporcionados por la plataforma. El proyecto de la calculadora no contiene archivos fuente Java propios.

Uso de cada lenguaje en la aplicación

- **HTML:** Definir la estructura del documento mediante index.html, incluyendo las secciones Historia, Modelo 3D y Cálculo; las pestañas de navegación; el visor de resultados; los indicadores de modo, memoria y operación pendiente; y los contenedores donde JavaScript incorpora las teclas numéricas y científicas. También proporciona las ventanas de diálogo utilizadas para presentar el manual, las fuentes documentales y los ajustes.

- **CSS:** Definir la presentación visual mediante styles.css: colores, tipografía local, tamaños, bordes, espaciado y distribución de los elementos. Organizar los bloques del teclado, el visor y los paneles de información, y adaptar su disposición a las dimensiones y orientación de la pantalla mediante reglas de diseño y consultas de medios. El ajuste de escala de la calculadora se complementa con mediciones realizadas desde JavaScript.

- **JavaScript:** Responder a las acciones del usuario y actualizar la interfaz. En app.js, generar los botones numéricos y científicos, detectar pulsaciones y entradas del teclado físico, cambiar entre las pestañas, mostrar el manual y los ajustes, y enviar las acciones a Kotlin mediante mensajes JSON a través de NativeApp.postMessage. Recibir las respuestas del código nativo y actualizar el resultado, los indicadores, los registros visibles y los mensajes de error. En scene.js, utilizar Three.js, GLTFLoader y OrbitControls para cargar y representar el modelo GLB, configurar la cámara y la iluminación, y permitir al usuario girar la vista, acercarla y restablecer su posición inicial. Las operaciones matemáticas de la calculadora se delegan al motor Kotlin.

- **Kotlin:** Implementar el motor de cálculo y la integración nativa con Android. En CalculatorEngine.kt, procesar la entrada numérica, las operaciones secuenciales, las funciones científicas, las fracciones, la conversión sexagesimal, la memoria y la estadística; comprobar errores como divisiones por cero, argumentos fuera de dominio y datos insuficientes. Mediante MainActivity.kt y AppView.kt, crear y configurar la WebView, cargar los recursos locales, recibir las solicitudes de JavaScript, comprobar su origen y devolver el estado actualizado en formato JSON. Gestionar también el portapapeles, el uso compartido de resultados, el sonido, la respuesta táctil y la persistencia de preferencias con SharedPreferences. Coordinar el ciclo de vida de la aplicación y la navegación hacia atrás con la interfaz

JavaScript, detener los sonidos al pausarse y liberar sus recursos al destruirse.

Bibliotecas y API utilizadas desde Kotlin

La aplicación utiliza librerías estándar de Kotlin y las API del Software Development Kit de Android. Una API es un conjunto de funciones y clases que permite utilizar las capacidades de una plataforma. (Android Developers, s. f.-e; JetBrains, s. f.-d)

- kotlin-stdlib: Biblioteca estándar de Kotlin. Reúne herramientas de uso general para trabajar con textos, números, colecciones de datos y errores. Facilita las tareas habituales de programación y ayuda a escribir el código de forma más sencilla y organizada (JetBrains, s. f.-d).

- android.app: Permite organizar las pantallas de una aplicación y gestionar su ciclo de vida. Esto incluye responder cuando una pantalla se abre, pasa a segundo plano, vuelve a mostrarse o se cierra (Android Developers, s. f.-e).

- android.webkit: Permite integrar contenido web dentro de una aplicación Android. Se encarga de mostrar HTML, aplicar CSS y ejecutar JavaScript, además de controlar qué contenido se carga y cómo se presenta (Android Developers, s. f.-a).

- android.view: Es un paquete del framework de Android que proporciona clases para gestionar las vistas nativas. En esta calculadora, permite configurar la integración del WebView en la pantalla, mientras que los elementos internos de la interfaz se definen y controlan mediante HTML, CSS y JavaScript (Android Developers, s. f.-e).

- android.widget: Ofrece componentes de interfaz preparados para utilizarse, como botones, campos de texto y contenedores. Los contenedores permiten agrupar y distribuir otros elementos dentro de una pantalla (Android Developers, s. f.-e).

- android.os: Permite trabajar con servicios e información del sistema operativo. Entre sus usos están consultar la versión de Android, intercambiar datos entre componentes y manejar información sobre el estado de una pantalla (Android Developers, s. f.-e).

- android.graphics: Se utiliza desde Kotlin para establecer el color de fondo del contenedor Android donde se muestra la calculadora. El diseño de los botones, los textos y la pantalla de resultados se define mediante HTML y CSS (Android Developers, s. f.-e).

- android.net: Permite examinar las direcciones de los recursos que intenta cargar el WebView. En esta calculadora ayuda a comprobar que se accede únicamente a los archivos locales incluidos en la aplicación (Android Developers, s. f.-e).

Estas API forman parte de la plataforma Android y sus implementaciones están disponibles en el dispositivo.

El Software Development Kit proporciona los elementos necesarios para compilar el código que las utiliza, por lo que no es necesario añadirlas individualmente como bibliotecas adicionales (Android Developers, s. f.-e).

Bibliotecas de JavaScript

La calculadora no utiliza bibliotecas externas de JavaScript. Las operaciones están implementadas en engine.js y la interacción con los botones está en app.js, utilizando JavaScript y las funciones disponibles en el WebView. No se utilizan React, React Native ni jQuery (artemioadaysolvers, s. f.).

La librería engine.js contiene la lógica de cálculo. Guarda los números y la operación seleccionada, realiza los cálculos y detecta errores como dividir entre cero. La app.js conecta los botones y la pantalla con esa lógica. Detecta las pulsaciones, envía las instrucciones a engine.js y muestra el resultado (artemioadaysolvers, s. f.).

Tutorial

Material para desarrollar la Aplicación .apk

- Un ordenador Windows de 64 bits que cumpla los requisitos de Android Studio (Android Developers, 2026).

- Conexión a Internet para descargar las herramientas y sincronizar el proyecto por primera vez.

- El proyecto fuente completo de la calculadora, versión 1.1.1.

- Un teléfono con Android 6.0 o posterior y con el componente Android System WebView disponible y actualizado. Android 15 está dentro del intervalo admitido por la configuración del proyecto (Android Developers, s. f.-f).

- Un sistema que permita transferir el .apk (app compilada) hacia el dispositivo Android.

El **APK** es el archivo instalable de la aplicación Android. El ZIP contiene los archivos necesarios para construir ese APK. Para probar la app utilizará el teléfono Android; no es necesario configurar un emulador (Android Developers, s. f.-b).

Descargar e instalar Android Studio

Las instrucciones de instalación y los requisitos del equipo se encuentran en la documentación oficial. (Android Developers, 2026)

1. Abre la página oficial de Android Studio: https://developer.android.com/studio

2. Descarga una versión estable para Windows compatible con Android Gradle Plugin 8.13.2. Elige el instalador con extensión .exe.

3. Ejecuta el instalador y sigue sus indicaciones. Puedes conservar la carpeta de instalación propuesta.

4. Abre Android Studio al terminar. Si pregunta si deseas importar una configuración anterior y es tu primera instalación, elige no importarla.

5. En el asistente inicial, selecciona la instalación **Standard** si está disponible. Si utilizas una configuración personalizada, incluye el Android SDK (Software Development Kit).

6. Revisa las licencias y acéptalas si estás de acuerdo. Espera a que termine la descarga de los componentes y pulsa **Finish**.

Descargar el repositorio de GitHub

Repositorio del taller:

https://github.com/artemioadaysolvers/App-Tutorial-I

1. Abre el enlace anterior en tu navegador. Si el repositorio requiere acceso, inicia sesión con una cuenta autorizada.

2. En la página del repositorio, pulsa **Code**.

3. Selecciona **Download ZIP**. Este procedimiento descarga una copia de los archivos de la rama seleccionada. (GitHub, s. f.)

4. Busca el ZIP en las descargas del ordenador.

5. Haz clic derecho y selecciona **Extraer todo**. Guarda los archivos en una carpeta local fácil de identificar, por ejemplo dentro de C:\Proyectos.

Abrir el proyecto en Android Studio

1. En la pantalla inicial de Android Studio, pulsa **Open**. Si ya tienes otro proyecto abierto, utiliza **File >Open**.

2. Busca la carpeta extraída del proyecto completo.

3. Selecciona **CasioFX2200-proyecto**, la carpeta que contiene los archivos de la aplicación.

4. Abre el proyecto y espera a que Android Studio termine de reconocer sus archivos.

Abre el proyecto extraído, no el archivo ZIP. No necesitas crear otro proyecto ni copiar los archivos dentro de una plantilla nueva.

Los archivos de configuración terminan en .gradle.kts porque utilizan Kotlin para configurar la compilación (JetBrains, s. f.-b).

Instalación y configuración del JDK y sus componentes en Android

Componente	Ubicación en Android Studio	Procedimiento de instalación y configuración
JDK 17 para Gradle	Abrir File, seleccionar Settings y entrar en Build, Execution, Deployment. Después, abrir Build Tools y seleccionar Gradle.	1. Desplegar el campo Gradle JDK. 2. Seleccionar Download JDK. 3. En Version, elegir 17. Conservar el proveedor y la ubicación de descarga propuestos por Android Studio. 4. Pulsar Download y esperar a que termine. 5. Comprobar que el JDK descargado quede seleccionado en Gradle JDK. Pulsar Apply y después OK.
Android SDK Platform 36	Abrir Tools, seleccionar SDK Manager y entrar en la pestaña SDK Platforms.	1. Activar Show Package Details. 2. Buscar y desplegar Android 16.0, correspondiente a API Level 36. 3. Marcar la casilla Android SDK Platform 36. Seleccionar únicamente ese paquete dentro de la sección. 4. Pulsar Apply, confirmar la instalación y aceptar las licencias cuando se soliciten. 5. Esperar a que termine y pulsar Finish.
Android SDK Build-Tools 35.0.0	Dentro de Tools y SDK Manager, seleccionar la pestaña SDK Tools.	1. Activar Show Package Details. 2. Buscar y desplegar Android SDK Build-Tools. 3. Dentro de la lista de versiones, marcar la casilla 35.0.0. 4. Pulsar Apply, confirmar la instalación y aceptar las licencias cuando se soliciten. 5. Esperar a que termine y pulsar Finish.
Android SDK Platform-Tools	Dentro de Tools y SDK Manager, permanecer en la misma pestaña SDK Tools.	1. Buscar Android SDK Platform-Tools. 2. Marcar su casilla para instalar la versión disponible. 3. Pulsar Apply, confirmar la instalación y aceptar las licencias cuando se soliciten. 4. Esperar a que termine y pulsar Finish. Finalmente, pulsar OK para cerrar el SDK Manager. Este paquete incluye ADB, utilizado para comunicar Android Studio con el teléfono mediante depuración USB.

Cuadro 9: Instalación y configuración del JDK para Gradle y de los componentes del SDK mediante Android Studio en Windows.

Sincronizar el proyecto

1. Si aparece **Sync Now**, pulsa esa opción. También puedes utilizar **File >Sync Project with Gradle Files**.

2. Mantén Internet conectado mientras Gradle descarga los componentes necesarios.

3. Si aparece **Install missing SDK packages**, revisa e instala los componentes indicados.

4. Espera a que termine la sincronización. Si falla, revisa el mensaje de error antes de continuar.

Gradle organiza la compilación y reúne los recursos de la aplicación. El proyecto incluye Gradle Wrapper, que selecciona y descarga la versión indicada en sus archivos. No hace falta instalar Gradle por separado. Para generar el APK de esta calculadora tampoco se requieren Node.js, npm ni Capacitor (Gradle, s. f.).

Conserva las versiones configuradas del proyecto durante esta primera compilación. Una sugerencia para actualizar herramientas no significa necesariamente que debas cambiarlas para generar el APK.

Generar el APK de la calculadora

1. Abre **View >Tool Windows >Build Variants** y selecciona **debug** para el módulo **app**.

2. En el menú superior, pulsa **Build**.

3. Selecciona **Generate Bundle(s) / APK(s)** y después **Generate APK(s)**. Según la versión de Android Studio, la primera opción también puede aparecer como **Generate App Bundles or APKs** o como **Build Bundle(s) / APK(s) >Build APK(s)**.

4. Espera a que finalice el proceso. La primera compilación puede tardar más porque descarga herramientas y prepara archivos.

5. Comprueba que el resultado indique BUILD SUCCESSFUL.

Para instalar manualmente en el teléfono, elige **APK**. El archivo con extensión .aab, llamado Android App Bundle, se utiliza en otro procedimiento de distribución (Android Developers, s. f.-b).

Para esta práctica, la variante debug ya queda firmada con una clave de desarrollo. No hace falta elegir **Generate Signed App Bundle or APK**; ese asistente permite preparar una versión con una clave de distribución propia. (Android Developers, s. f.-b)

Encontrar y guardar el archivo correcto

Al terminar, pulsa **Locate** en la notificación de Android Studio si aparece. También puedes buscar el resultado desde el Explorador de archivos de Windows, dentro de la carpeta del proyecto:

CasioFX2200-proyecto\app\build\outputs\apk\debug\app-debug.apk

El archivo que debes copiar al teléfono se llama **app-debug.apk**. Puedes guardar una copia en Descargas del ordenador o en otra carpeta que identifiques fácilmente (Android Developers, s. f.-b).

BUILD SUCCESSFUL confirma que la compilación terminó correctamente. Después debes instalar y abrir la aplicación para comprobar su funcionamiento en el teléfono (Gradle, s. f.).

- index.html: estructura de la pantalla y botones.
- styles.css: presentación y tamaños.
- engine.js: operaciones y estado de la calculadora.
- app.js: interacción de los botones y el teclado.

La parte nativa de Android está en:

app/src/main/kotlin/com/retrolab/casiofx2200/MainActivity.kt

El archivo MainActivity.kt crea la ventana, configura el WebView, carga los archivos locales y gestiona las pausas y el cierre de la aplicación. Los cálculos continúan implementados en JavaScript (artemioadaysolvers, s. f.).

Otros archivos útiles son:

- app/src/main/res/values/strings.xml: nombre visible de la aplicación.
- app/src/main/AndroidManifest.xml: configuración principal de Android y declaración de la actividad.
- app/build.gradle.kts: identificador de instalación, versiones y niveles del SDK.

Referencias

Android Developers. (s. f.-a). *Build web apps in WebView*. https://developer.android.com/develop/ui/views/layout/webapps/webview

Android Developers. (s. f.-b). *Build your app for release to users*. https://developer.android.com/build/build-for-release

Android Developers. (s. f.-c). *Java versions in Android builds*. https://developer.android.com/build/jdks

Android Developers. (s. f.-d). *Manage and diagnose WebView memory*. https://developer.android.com/develop/ui/views/layout/webapps/manage-webview-memory

Android Developers. (s. f.-e). *Package index*. https://developer.android.com/reference/packages

Android Developers. (s. f.-f). *Uses-sdk*. https://developer.android.com/guide/topics/manifest/uses-sdk-element

Android Developers. (2025). *Android Gradle plugin 8.13.0 (September 2025)*. https://developer.android.com/build/releases/agp-8-13-0-release-notes

Android Developers. (2026). *Install Android Studio*. https://developer.android.com/studio/install

artemioadaysolvers. (s. f.). *App-Tutorial-I* [Repositorio de código]. GitHub. https://github.com/artemioadaysolvers/App-Tutorial-I

Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G. (2019). Animations in cross-platform mobile applications: An evaluation of tools, metrics and performance. *Sensors, 19*(9), 2081. https://doi.org/10.3390/s19092081

Capacitor. (s. f.). *Capacitor documentation*. https://capacitorjs.com/docs

GitHub. (s. f.). *Downloading source code archives*. https://docs.github.com/en/repositories/working-with-files/using-files/downloading-source-code-archives

Gradle. (s. f.). *Gradle Wrapper*. https://docs.gradle.org/current/userguide/gradle_wrapper.html

Harrelson, C. (2021). *RenderingNG architecture*. Chrome for Developers. https://developer.chrome.com/docs/chromium/renderingng-architecture

Huber, S., Demetz, L., & Felderer, M. (2020). Analysing the performance of mobile cross-platform development approaches using UI interaction scenarios. En *Software technologies* (pp. 40-57). Springer. https://doi.org/10.1007/978-3-030-52991-8_3

Ionic. (s. f.). *Ionic Framework*. https://ionicframework.com/docs

JetBrains. (s. f.-a). *Calling Java from Kotlin*. https://kotlinlang.org/docs/java-interop.html

JetBrains. (s. f.-b). *Configure a Gradle project*. https://kotlinlang.org/docs/gradle-configure-project.html

JetBrains. (s. f.-c). *FAQ*. https://kotlinlang.org/docs/faq.html

JetBrains. (s. f.-d). *kotlin-stdlib: Core API*. https://kotlinlang.org/api/core/kotlin-stdlib/

MDN Web Docs. (s. f.). *Aprende desarrollo web*. https://developer.mozilla.org/es/docs/Learn_web_development

Oliveira, W., Moraes, B., Castor, F., & Fernandes, J. P. (2023). Analyzing the resource usage overhead of mobile app development frameworks. En *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering* (pp. 152-161). Association for Computing Machinery. https://doi.org/10.1145/3593434.3593487

Oracle. (2025). *The Java Virtual Machine Specification, Java SE 25 edition: Chapter 1. Introduction*. https://docs.oracle.com/javase/specs/jvms/se25/html/jvms-1.html

React. (s. f.). *Describing the UI*. https://react.dev/learn/describing-the-ui

React Native. (s. f.-a). *Animations*. https://reactnative.dev/docs/animations

React Native. (s. f.-b). *Architecture overview*. https://reactnative.dev/architecture/overview

React Native. (s. f.-c). *Core components and APIs*. https://reactnative.dev/docs/components-and-apis

React Native. (s. f.-d). *Fabric*. https://reactnative.dev/architecture/fabric-renderer

React Native. (s. f.-e). *Introduction*. https://reactnative.dev/docs/getti

ng-started

Android Developers. (s. f.-a). *Build web apps in WebView*. https://developer.android.com/develop/ui/views/layout/webapps/webview

Android Developers. (s. f.-b). *WebView*. https://developer.android.com/reference/kotlin/android/webkit/WebView

Apple. (s. f.). *Programming with Objective-C*. https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/

Ecma International. (2026). *ECMAScript 2026 language specification (ECMA-262, 17th ed.)*. https://262.ecma-international.org/

ISO C++ Foundation. (s. f.). *The standard*. https://isocpp.org/std/the-standard

Kotlin. (s. f.). *Kotlin for Android*. https://kotlinlang.org/docs/android-overview.html

Microsoft. (s. f.). *TypeScript for the new programmer*. https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html

Oracle. (2026). *Java language updates: Release 25*. https://docs.oracle.com/en/java/javase/25/language/index.html

React Native. (2024). *New architecture is here*. https://reactnative.dev/blog/2024/10/23/the-new-architecture-is-here

React Native. (2026). *Using TypeScript*. https://reactnative.dev/docs/typescript

React Native. (s. f.-a). *Cross-platform native modules (C++)*. https://reactnative.dev/docs/next/the-new-architecture/pure-cxx-modules

React Native. (s. f.-b). *iOS: Using Swift in your native modules*. https://reactnative.dev/docs/0.86/the-new-architecture/turbo-modules-with-swift

React Native. (s. f.-c). *JavaScript environment*. https://reactnative.dev/docs/javascript-environment

React Native. (s. f.-d). *Turbo native modules: Android*. https://reactnative.dev/docs/turbo-native-modules-android

Swift. (s. f.). *About Swift*. https://www.swift.org/about/

Android Developers. (2025). *Use Java 8 language features and APIs*. https://developer.android.com/studio/write/java8-support

Android Developers. (2026a). *AGP, D8, and R8 versions required for Kotlin versions*. https://developer.android.com/build/kotlin-support

Android Developers. (2026b). *Android build structure*. https://developer.android.com/build/android-build-structure

Android Developers. (2026c). *Android KTX*. https://developer.android.com/kotlin/ktx

Android Developers. (2026d). *Android's Kotlin-first approach*. https://developer.android.com/kotlin/first

Android Developers. (2026e). *Asynchronous background processing*. https://developer.android.com/develop/background-work/background-tasks/asynchronous

Android Developers. (2026f). *Kotlin for Jetpack Compose*. https://developer.android.com/develop/ui/compose/kotlin

Android Developers. (2026g). *Kotlin on Android FAQ*. https://developer.android.com/kotlin/faq

Android Developers. (2026h). *Layouts in views*. https://developer.android.com/develop/ui/views/layout/declaring-layout

artemioadaysolvers. (s. f.). *App-Tutorial-I* [Repositorio de código]. GitHub. https://github.com/artemioadaysolvers/App-Tutorial-I

Android Developers. (2026a). *Configure your build*. https://developer.android.com/build

Android Developers. (2026b). *Gradle build overview*. https://developer.android.com/build/gradle-build-overview

Gradle. (s. f.-a). *Build file basics. Gradle User Manual*. https://docs.gradle.org/current/userguide/build_file_basics.html

Gradle. (s. f.-b). *Build lifecycle. Gradle User Manual*. https://docs.gradle.org/current/userguide/build_lifecycle.html

Gradle. (s. f.-c). *Core concepts. Gradle User Manual*. https://docs.gradle.org/current/userguide/gradle_basics.html

Gradle. (s. f.-d). *Dependencies and dependency management basics. Gradle User*

Manual. https://docs.gradle.org/current/userguide/dependency_management_basics.html

Gradle. (s. f.-e). *Gradle Wrapper. Gradle User Manual*. https://docs.gradle.org/current/userguide/gradle_wrapper.html

Android Developers. (2025). *Update the IDE and SDK tools*. https://developer.android.com/studio/intro/update

Android Developers. (2026a). *Build and run your app in Android Studio*. https://developer.android.com/build/building-studio

Android Developers. (2026b). *Configure your build*. https://developer.android.com/build

Android Developers. (2026c). *Gradle build overview*. https://developer.android.com/build/gradle-build-overview

Gradle. (s. f.). *Build lifecycle. Gradle User Manual*. https://docs.gradle.org/current/userguide/build_lifecycle.html

Android Developers. (2026a). *Build and run your app*. https://developer.android.com/studio/run

Android Developers. (2026b). *Build and run your app in Android Studio*. https://developer.android.com/build/building-studio

Android Developers. (2026c). *Build your app from the command line*. https://developer.android.com/build/building-cmdline

Android Developers. (2026d). *Configure build variants*. https://developer.android.com/build/build-variants

Android Developers. (2026e). *Configure your build*. https://developer.android.com/build

Android Developers. (2026f). *Gradle build overview*. https://developer.android.com/build/gradle-build-overview

Gradle. (s. f.). *Build lifecycle. Gradle User Manual*. https://docs.gradle.org/current/userguide/build_lifecycle.html

Android Developers. (2026a, 26 de febrero). *Build your app from the command line*. https://developer.android.com/build/building-cmdline

Android Developers. (2026b, 5 de marzo). *Save simple data with SharedPreferences*. https://developer.android.com/training/data-storage/shared-prefere

nces

Android Developers. (2026c, 3 de agosto). *SoundPool*. https://developer.android.com/reference/android/media/SoundPool

Android Developers. (2026d, 9 de septiembre). *WebViewCompat*. https://developer.android.com/reference/androidx/webkit/WebViewCompat

Android Developers. (2026e, 16 de octubre). *Build web apps in WebView*. https://developer.android.com/develop/ui/views/layout/webapps/webview

Android Developers. (2026f, 16 de octubre). *Configure your build*. https://developer.android.com/build

Android Developers. (2026g, 19 de octubre). *Load in-app content*. https://developer.android.com/develop/ui/views/layout/webapps/load-local-content

Android Developers. (2026h, 19 de octubre). *Send data to other apps*. https://developer.android.com/develop/ui/compose/sharing/send

GitHub. (s. f.). *Adding a repository from your local computer to GitHub Desktop*. https://docs.github.com/en/desktop/adding-and-cloning-repositories/adding-a-repository-from-your-local-computer-to-github-desktop

JetBrains. (2026, 18 de junio). *Numbers*. https://kotlinlang.org/docs/numbers.html

Three.js. (s. f.-a). *GLTFLoader*. https://threejs.org/docs/pages/GLTFLoader.html

Three.js. (s. f.-b). *OrbitControls*. https://threejs.org/docs/pages/OrbitControls.html

Android Developers. (2026a, 26 de febrero). *Build your app from the command line*. https://developer.android.com/build/building-cmdline

Android Developers. (2026b, 18 de mayo). *App resources overview*. https://developer.android.com/guide/topics/resources/providing-resources

Android Developers. (2026c, 9 de septiembre). *WebViewCompat*. https://developer.android.com/reference/androidx/webkit/WebViewCompat

Android Developers. (2026d, 25 de septiembre). *Android Debug Bridge (adb)*. https://developer.android.com/tools/adb

Android Developers. (2026e, 16 de octubre). *App manifest overview*. https://

developer.android.com/guide/topics/manifest/manifest-intro

Android Developers. (2026f, 16 de octubre). *Configure your build*. https://developer.android.com/build

Android Developers. (2026g, 16 de octubre). *Java versions in Android builds*. https://developer.android.com/build/jdks

Android Developers. (2026h, 16 de octubre). *SDK Build Tools release notes*. https://developer.android.com/tools/releases/build-tools

Android Developers. (2026i, 16 de octubre). *SDK Platform release notes*. https://developer.android.com/tools/releases/platforms

Android Developers. (2026j, 16 de octubre). *SDK Platform Tools release notes*. https://developer.android.com/tools/releases/platform-tools

Android Developers. (2026k, 19 de octubre). *Load in-app content*. https://developer.android.com/develop/ui/views/layout/webapps/load-local-content

Android Developers. (2026l, 21 de octubre). *Android Gradle Plugin 8.12.0 (July 2025)*. https://developer.android.com/build/releases/agp-8-12-0-release-notes

JetBrains. (2026, 6 de octubre). *Configure a Gradle project*. https://kotlinlang.org/docs/gradle-configure-project.html

Oracle. (s. f.). *Java SE 17 archive downloads*. https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html

Google. (s. f.). *Android Studio* [Repositorio de código fuente]. Git at Google. Recuperado el 26 de octubre de 2026, de https://android.googlesource.com/platform/tools/adt/idea/

Google. (2025a, 27 de julio). *Arquitectura de la plataforma*. Android Developers. https://developer.android.com/guide/platform?hl=es-419

Google. (2025b, 16 de septiembre). *Cómo preparar tu app para el lanzamiento*. Android Developers. https://developer.android.com/studio/publish/preparing?hl=es-419

Google. (2025c, 27 de julio). *d8*. Android Developers. https://developer.android.com/tools/d8?hl=es-419

Google. (2026a, 17 de febrero). *Actualiza el IDE y las herramientas del SDK*. Android Developers. https://developer.android.com/studio/intro/upd

ate?hl=es-419

Google. (2026b, 6 de marzo). *Cómo firmar tu app*. Android Developers. https://developer.android.com/studio/publish/app-signing?hl=es-419

Google. (2026c, 17 de octubre). *Descripción general de la compilación de Gradle*. Android Developers. https://developer.android.com/build/gradle-build-overview?hl=es-419

Google. (2026d, 2 de octubre). *Introducción a Android Studio*. Android Developers. https://developer.android.com/studio/intro?hl=es-419

JetBrains. (s. f.). *IntelliJ IDEA: The Java and Kotlin IDE for you and your agents*. Recuperado el 26 de octubre de 2026, de https://www.jetbrains.com/idea/